Jour 2 : fin du module 1, Zone.js, RxJS, guards, signaux

Copier un code : cliquer dans la case blanche à droite du titre du bloc, puis **Cmd+A** et **Cmd+C** (sous Windows : **Ctrl+A** et **Ctrl+C**). Le code copié est exact, avec son indentation.

Plan de la journée

Horaire	Partie
9 h 00 – 9 h 10	Reprise de la formation
9 h 10 – 9 h 25	Nx : ne refaire que le nécessaire
9 h 25 – 9 h 35	Les bibliothèques : réutiliser le code
9 h 35 – 9 h 55	Charger le code à la demande
9 h 55 – 10 h 15	Des images qui ne se téléchargent qu'au besoin
10 h 15 – 10 h 20	Documenter le code avec Compodoc
10 h 20 – 10 h 40	Ne pas faire travailler l'affichage pour rien
10 h 40 – 10 h 55	pause
10 h 55 – 11 h 15	Zone.js : ce qui a changé
11 h 15 – 11 h 40	RxJS : partager une requête
11 h 40 – 12 h 10	RxJS : une recherche qui attend la fin de la frappe
12 h 10 – 12 h 30	RxJS : enchaîner deux requêtes sans les imbriquer
12 h 30 – 13 h 30	déjeuner
13 h 30 – 13 h 50	RxJS : arrêter une minuterie (fuite de mémoire)
13 h 50 – 14 h 10	Réserver des pages aux personnes connectées
14 h 10 – 14 h 25	Charger les données avant d'afficher la page
14 h 25 – 15 h 00	Les signaux : un état partagé qui se met à jour seul
15 h 00 – 15 h 35	Signaux et RxJS : passer de l'un à l'autre
15 h 35 – 15 h 50	pause
15 h 50 – 17 h 00	TP Cuisto, jour 2
Si le temps le permet	Pour aller plus loin

9 h 00 – 9 h 10

Reprise de la formation

Tout le monde repart du même code, avec Nx et les bibliothèques déjà en place.

> **Reprise (10 min)**
>
> **Hier** : introduction, visite et diagnostic de ShopFlow, installation de Nx, création des bibliothèques.
>
> **Ce matin, tout le monde repart du même code**, avec Nx et les bibliothèques déjà en place, pour ShopFlow et pour Cuisto. Personne ne refait les commandes d'hier : on regarde le résultat, puis on passe à la suite du programme.

Repartir des points de reprise

Le formateur distribue `shopflow-debut-jour-2.zip` et `cuisto-debut-jour-2.zip` (dossier `ressources/points-de-reprise/`). ShopFlow y est déjà passé sous Nx et découpé en bibliothèques, sans les corrections de ce matin : on les fait ensemble. On met son propre code de côté, puis on dézippe :

Terminal — dans le dossier qui contient formation-angular/ `cd formation-angular`

```
cd formation-angular
mv shopflow shopflow-mon-jour-1
mv cuisto cuisto-mon-jour-1
unzip ~/Downloads/shopflow-debut-jour-2.zip
unzip ~/Downloads/cuisto-debut-jour-2.zip
```

Puis on installe et on lance chaque application dans son terminal :

Terminal 1 — ShopFlow (http://localhost:4200) `cd formation-angular/shopflow`

```
cd formation-angular/shopflow
npm install
npm start
```

Terminal 2 — Cuisto (http://localhost:4201) `cd formation-angular/cuisto`

```
cd formation-angular/cuisto
npm install
npm start -- --port 4201
```

9 h 10 – 9 h 25

Nx : ne refaire que le nécessaire

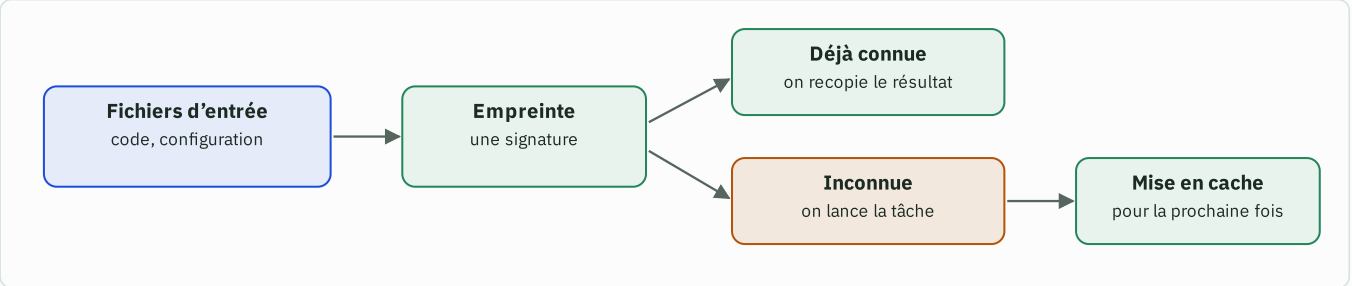
Module 1, « Introduction au CLI Nx ».

1. La notion

Objectif : lancer les tâches de ShopFlow avec Nx et comprendre ce que son cache fait gagner.

Pourquoi : avec Angular CLI seul, chaque construction refait tout, même si rien n'a changé.

Dans la réalité : une équipe pousse des dizaines de modifications par jour, et l'intégration continue reconstruit et reteste le projet à chaque fois. Sur un gros dépôt, cela prend des dizaines de minutes ; avec le cache de Nx, ce qui n'a pas changé n'est pas refait.



Nx calcule une signature des fichiers d'entrée ; s'il la connaît déjà, il recopie le résultat au lieu de refaire la tâche.

Nx Console (l'extension de VS Code) lance exactement les mêmes commandes, avec la souris et un formulaire. On l'utilise quand on ne connaît pas les options d'une commande ; en intégration continue, il n'y a que la ligne de commande. On choisit l'un ou l'autre, jamais les deux.

2. La démonstration

On construit deux fois de suite, sans rien modifier.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) `npx nx build shopflow`

```
npx nx build shopflow
npx nx build shopflow
```

3. La mise en œuvre

Rien à corriger ici : on change seulement sa façon de lancer les tâches.

- `npx nx serve shopflow`, `npx nx build shopflow`, `npx nx test shopflow` à la place de `ng serve`, `ng build`, `ng test`.
- Modifier un texte dans `src/app/pages/catalogue/catalogue.html`, reconstruire : la tâche est refaite. Annuler la modification, reconstruire : le cache resservit le résultat.

4. Le résultat

Terminal : shopflow

```
$ npx nx build
 NX   Successfully ran target build for project shopflow
  Run duration:      7.8s
  Cache:             0/1 hit (0%)
  Critical path:     7.5s (1 task)
  Recoverable time:  <1ms
$ npx nx build
 NX   Successfully ran target build for project shopflow
Nx read the output from the cache instead of running the command for 1 out of 1 tasks.
  Run duration:      11ms
  Cache:             1/1 hit (100%)
  Critical path:     2ms (1 task)
  Recoverable time:  <1ms
```

Première construction : 7,8 s. Deuxième : 11 ms, « read the output from the cache ».

9 h 25 – 9 h 35

Les bibliothèques : réutiliser le code

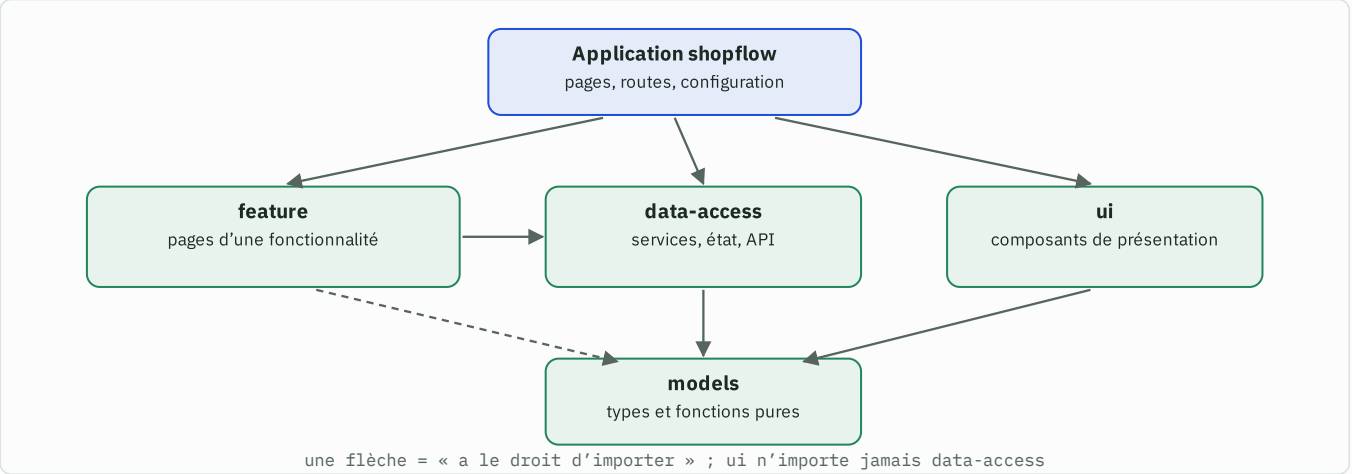
Module 1, « Architecture applicative ». Démonstration sur le point de reprise : rien à taper.

1. La notion

Objectif : comprendre pourquoi ShopFlow est découpé en trois bibliothèques, et lire ce découpage.

Pourquoi : au départ, la carte produit injectait le panier : impossible de la réutiliser ailleurs, et personne ne savait ce que touchait une modification.

Dans la réalité : l'entreprise veut un back-office pour les vendeurs, qui affiche les mêmes cartes produit. Sans bibliothèque, on copie le fichier, et les deux copies finissent par différer. Avec une bibliothèque `@shopflow/ui`, les deux applications importent la même carte.



Les quatre types de bibliothèques recommandés par Nx. ShopFlow utilise data-access, ui et models ; ses pages restent dans l'application pour l'instant.

2. La démonstration

Ouvrir le point de reprise dans VS Code : le dossier `libs/shared/` contient trois bibliothèques. **models** : les types (un produit, une commande) ; **data-access** : les services qui parlent à l'API et gardent l'état ; **ui** : les composants d'affichage, qui ne connaissent aucun service. L'application les importe par leur alias, par exemple `@shopflow/ui`.

3. Ce qui a été fait

Le découpage est déjà fait dans le point de reprise ; les commandes sont en annexe, pour qui veut les lire. La seule partie qui demande de réfléchir : **la carte ne décide plus de ce qui se passe au clic**. Elle émet un événement, et c'est la page qui appelle le panier.

libs/shared/ui/src/lib/product-card/product-card.ts — `import { Component, input, output } from`

```ts
import { Component, input, output } from '@angular/core';
import { CurrencyPipe, DecimalPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Product, prixRemise } from '@shopflow/models';

@Component({
  selector: 'sf-product-card',
  imports: [CurrencyPipe, DecimalPipe, RouterLink],
  templateUrl: './product-card.html',
})
export class ProductCard {
  readonly product = input.required<Product>();
  readonly ajout = output<Product>();

  protected prixFinal(): number {
    return prixRemise(this.product());
  }

  protected ajouter(): void {
    this.ajout.emit(this.product());
  }
}
```

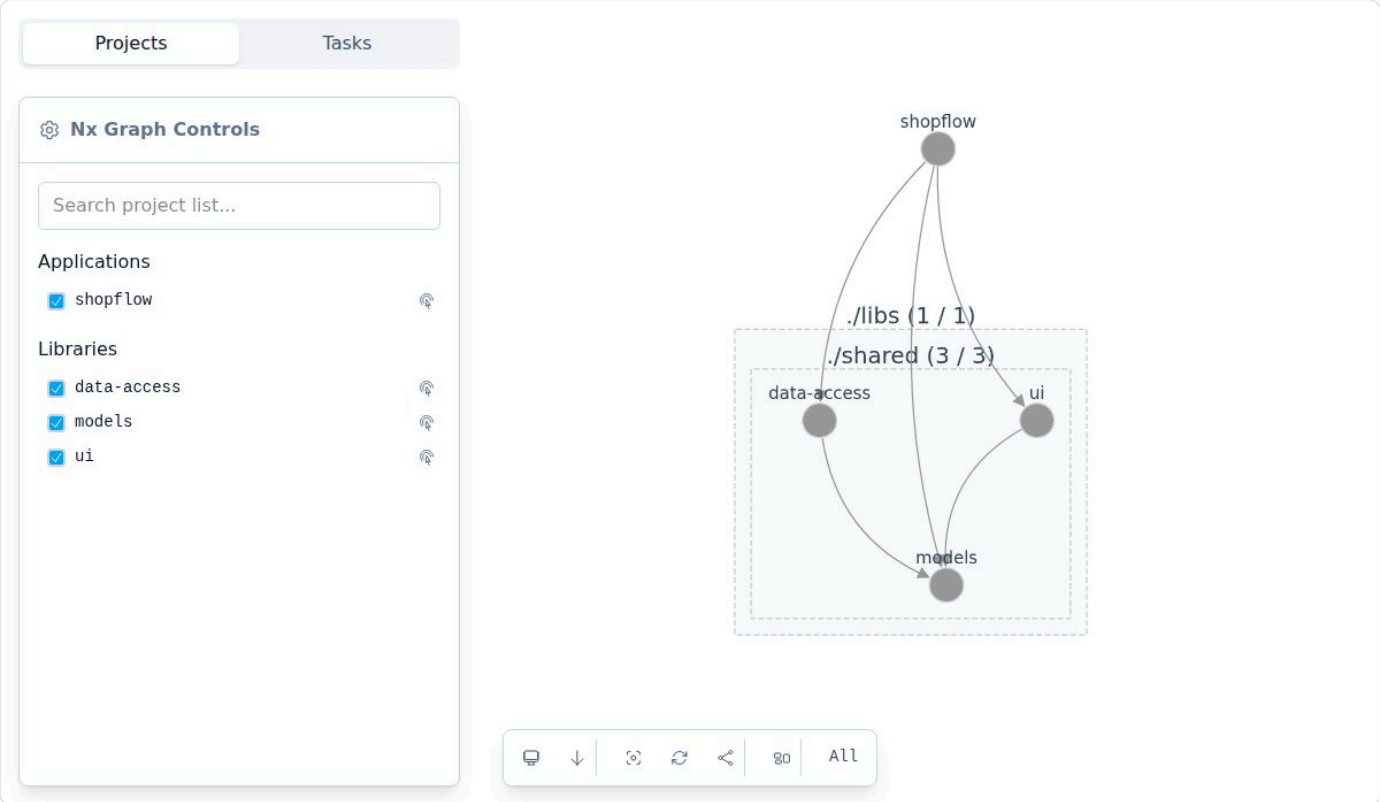
src/app/pages/catalogue/catalogue.html (extrait) (extrait) — `@for (p of`

```
@for (p of produitsAffiches(page.products); track $index) {
  <sf-product-card [product]="p" (ajout)="panier.add($event)" />
}
```

4. Le résultat

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) — `npx nx graph`

```
npx nx graph
```



Le graphe : l'application dépend des trois bibliothèques ; ui et data-access dépendent de models ; ui ne dépend pas de data-access.

Ce que montre le graphe : si l'on modifie la carte (`ui`), Nx sait qu'il faut reconstruire et retester l'application, mais pas les services.

9 h 35 – 9 h 55

Charger le code à la demande

Module 1, « Component-level lazy loading ».

1. La notion

Objectif : ne télécharger que le code de la page affichée, et ne charger une partie de page que lorsqu'elle devient visible.

Pourquoi : au départ, le visiteur du catalogue télécharge aussi le code de l'administration, du panier, de la commande et des avis.

Dans la réalité : un client sur son téléphone, en 4G, attend que tout soit téléchargé avant de voir le premier produit. Au-delà de trois secondes, une bonne partie des visiteurs quittent le site.

Deux outils : une **route chargée à la demande** (`loadComponent`, `loadChildren`) télécharge le code d'une page quand on y va ; un **bloc** `@defer` dans un template ne télécharge et n'affiche son contenu qu'à un moment choisi, par exemple quand il arrive à l'écran.

2. La démonstration

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx build shopflow

```
npx nx build shopflow
```

À la fin de la construction, la liste ne contient que des « Initial chunk files » : tout le code est téléchargé dès l'ouverture.

3. La correction

Les routes :

src/app/app.routes.ts

```
import { Routes } from '@angular/router';
```

```ts
import { Routes } from '@angular/router';
import { Catalogue } from './pages/catalogue/catalogue';

export const routes: Routes = [
  { path: '', redirectTo: 'catalogue', pathMatch: 'full' },
  { path: 'catalogue', component: Catalogue },
  {
    path: 'catalogue/:id',
    loadComponent: () => import('./pages/product-detail/product-detail').then((m) => m.ProductDetail),
  },
  { path: 'panier', loadComponent: () => import('./pages/cart/cart').then((m) => m.Cart) },
  { path: 'commande', loadComponent: () => import('./pages/checkout/checkout').then((m) => m.Checkout) },
  { path: 'commande/confirmation', loadComponent: () => import('./pages/checkout/confirmation').then((m) =>
m.Confirmation) },
  { path: 'connexion', loadComponent: () => import('./pages/login/login').then((m) => m.Login) },
  { path: 'mes-commandes', loadComponent: () => import('./pages/orders/orders').then((m) => m.Orders) },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes').then((m) => m.adminRoutes) },
  { path: '**', redirectTo: 'catalogue' },
];
```

src/app/admin/admin.routes.ts

```
import { Routes } from '@angular/router';
```

```ts
import { Routes } from '@angular/router';
import { AdminLayout } from './admin-layout';
import { AdminDashboard } from './admin-dashboard';
import { StockTable } from './stock-table';

export const adminRoutes: Routes = [
  {
    path: '',
    component: AdminLayout,
    children: [
      { path: '', component: AdminDashboard },
      { path: 'stock', component: StockTable },
    ],
  },
];
```

Les avis de la fiche produit, différés :

src/app/pages/product-detail/product-reviews.ts

```
import { Component, input } from
```

```ts
import { Component, input } from '@angular/core';
import { DatePipe } from '@angular/common';
import { Review } from '@shopflow/models';
```

```ts
@Component({
  selector: 'app-product-reviews',
  imports: [DatePipe],
  template: `
    <ul class="avis-liste">
      @for (a of avis(); track $index) {
        <li class="avis-item">
          <p><strong>{{ a.reviewerName }}</strong> : {{ a.rating }}/5, le {{ a.date | date: 'longDate' }}</p>
          <p>{{ a.comment }}</p>
        </li>
      }
    </ul>
  `,
})
export class ProductReviews {
  readonly avis = input.required<Review[]>();
}
```

src/app/pages/product-detail/product-detail.html

```
<a class="retour" routerLink="/catalogue">← Retour au catalogue</a>
@if (produit$ | async; as produit) {
  <article class="detail">
    <img class="detail-image" [ngSrc]="produit.thumbnail" [alt]="produit.title" width="300" height="300"
priority />
    <div>
      <p class="carte-categorie">{{ produit.category }}</p>
      <h1>{{ produit.title }}</h1>
      <sf-note [valeur]="produit.rating" />
      <p class="detail-prix">{{ produit.price | currency: 'EUR' }}</p>
      <p>{{ produit.description }}</p>
      <p class="info">Stock : {{ produit.stock }}</p>
      <button class="bouton" type="button" (click)="ajouter(produit)">Ajouter au panier</button>
    </div>
  </article>
  <section class="avis">
    <h2>Avis clients</h2>
    @defer (on viewport) {
      <app-product-reviews [avis]="produit.reviews" />
    } @placeholder {
      <p class="info">Les avis s'affichent quand vous arrivez ici.</p>
    } @loading (minimum 300ms) {
      <p class="info">Chargement des avis…</p>
    }
  </section>
} @else {
  <p class="info">Chargement du produit…</p>
}
```

4. Le résultat

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx build shopflow

```
npx nx build shopflow
```

Terminal : shopflow

```
$ npx nx build
Initial chunk files | Names           |  Raw size | Estimated transfer size
chunk-jTV0udJV.js   | -               | 192.13 kB |               56.54 kB
main-K5L6C4XQ.js    | main            | 102.84 kB |               27.00 kB
chunk-DidGQIlI.js   | -               |  19.43 kB |                5.80 kB
styles-SZZ3P6BQ.css | styles          |   6.36 kB |                1.60 kB
                    | Initial total   | 320.76 kB |               90.94 kB
Lazy chunk files    | Names           |  Raw size | Estimated transfer size
chunk-BIykb_RY.js   | -               |  35.33 kB |                8.16 kB
chunk-CGIg2IYU.js   | admin-routes    |   3.88 kB |                1.48 kB
chunk-BnCr3Vsp.js   | cart            |   2.83 kB |                1.21 kB
chunk-DSp8CoHU.js   | checkout        |   2.78 kB |                1.03 kB
chunk-DHVlDHTu.js   | product-detail  |   2.30 kB |                1.15 kB
chunk-DwbAeuyh.js   | orders          |   2.24 kB |              985 bytes
chunk-CAf7glEE.js   | login           |   2.03 kB |              932 bytes
chunk-CiLaO77V.js   | live-orders     |   1.41 kB |              765 bytes
chunk-DZnQ5jM8.js   | confirmation    | 830 bytes |              830 bytes
chunk-RAD1ecu3.js   | product-reviews | 782 bytes |              782 bytes
Application bundle generation complete. [5.461 seconds] - 2026-10-15T14:52:41.755Z
Output location: /home/claude/sf3/shopflow/dist/shopflow
```

Les pages sont dans des « Lazy chunk files » ; product-reviews et live-orders ont leur propre fichier grâce à @defer.

Dans l'onglet Réseau, sur une fiche produit : le code et la requête des avis ne partent qu'en descendant jusqu'au bloc.

9 h 55 – 10 h 15

Des images qui ne se téléchargent qu’au besoin

Module 1, « TP : analyser et optimiser une application ».

1. La notion

Objectif : ne télécharger que les images visibles, et la plus importante en premier.

Pourquoi : à l’ouverture du catalogue, les 194 images sont téléchargées, alors que l’écran n’en montre que quelques-unes.

Dans la réalité : c’est la première cause de lenteur des boutiques en ligne : des mégaoctets d’images que personne ne voit, payés sur le forfait mobile du client.

La directive `NgOptimizedImage` remplace `src` par `ngSrc` et demande la largeur et la hauteur : les images hors de l’écran attendent qu’on s’en approche, et l’attribut `priority` fait télécharger en premier l’image la plus visible.

2. La démonstration

F12, onglet **Réseau**, bouton **Img**, recharger le catalogue : 194 images.

3. La correction

libs/shared/ui/src/lib/product-card/product-card.ts — import { Component, input, OnInit, output

```ts
import { Component, input, OnInit, output } from '@angular/core';
import { CurrencyPipe, NgOptimizedImage } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Product, prixRemise } from '@shopflow/models';
import { Note } from '../note/note';

@Component({
  selector: 'sf-product-card',
  imports: [CurrencyPipe, NgOptimizedImage, Note, RouterLink],
  templateUrl: './product-card.html',
})
export class ProductCard implements OnInit {
  readonly product = input.required<Product>();
  readonly prioritaire = input(false);
  readonly ajout = output<Product>();

  protected prioriteInitiale = false;

  ngOnInit(): void {
    this.prioriteInitiale = this.prioritaire();
  }

  protected prixFinal(): number {
    return prixRemise(this.product());
```

```
  }

  protected ajouter(): void {
    this.ajout.emit(this.product());
  }
}
```

libs/shared/ui/src/lib/product-card/product-card.html

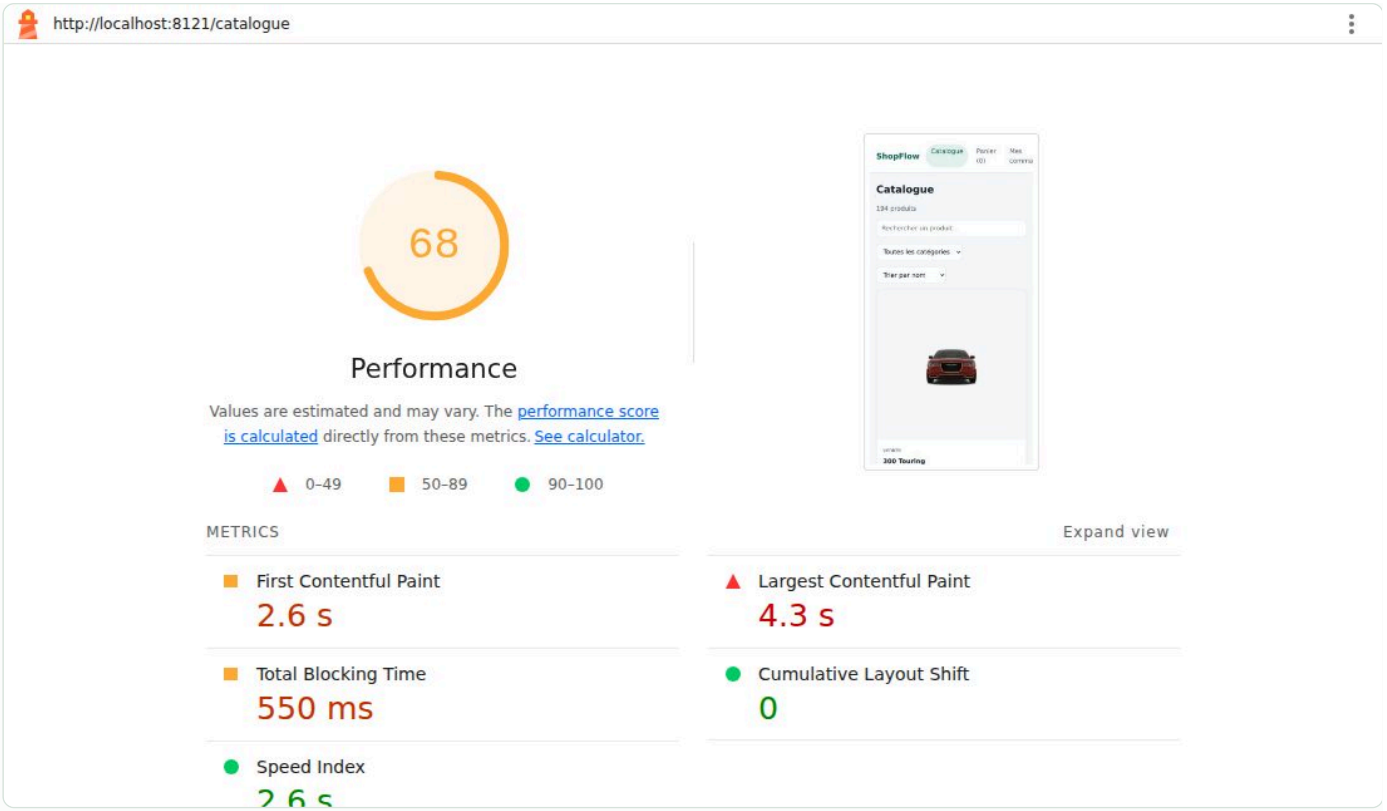
```
@let p = product();
<article class="carte">
  <a [routerLink]="['/catalogue', p.id]">
    <img class="carte-image" [ngSrc]="p.thumbnail" [alt]="p.title" width="300" height="300"
[priority]="prioriteInitiale" />
  </a>
  <div class="carte-corps">
    <p class="carte-categorie">{{ p.category }}</p>
    <h2 class="carte-titre"><a [routerLink]="['/catalogue', p.id]">{{ p.title }}</a></h2>
    <sf-note [valeur]="p.rating" />
    <p class="carte-prix">
      {{ prixFinal() | currency: 'EUR' }}
      @if (p.discountPercentage > 0) {
        <s class="prix-barre">{{ p.price | currency: 'EUR' }}</s>
      }
    </p>
    <button class="bouton" type="button" (click)="ajouter()">Ajouter au panier</button>
  </div>
</article>
```

src/app/pages/catalogue/catalogue.html (extrait) (extrait)

```
<sf-product-card [product]="p" [prioritaire]="$index < 4" (ajout)="panier.add($event)" />
```

4. Le résultat

Mesure	Avant	Après
Images téléchargées à l'ouverture	194	32
Requêtes au chargement (Lighthouse, mobile)	199	14
Poids de la page	2 121 Kio	420 Kio
Lighthouse mobile	42	68



Lighthouse en mobile après la séquence : 68 sur 100 (42 au départ).

10 h 15 – 10 h 20

Documenter le code avec Compodoc

Module 1, « Documentation avec Compodoc ».

1. La notion

Objectif : produire automatiquement un site qui décrit les composants, les services et les routes.

Pourquoi : ShopFlow n'a aucune documentation : un nouveau développeur doit tout lire pour comprendre.

Dans la réalité : à chaque arrivée dans l'équipe, on gagne des jours si l'on peut parcourir la liste des composants, de leurs entrées et de leurs sorties.

2. La démonstration

Aucune documentation dans le projet.

3. La mise en œuvre

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) `npm install --save-dev @compodoc/compodoc`

```
npm install --save-dev @compodoc/compodoc
```

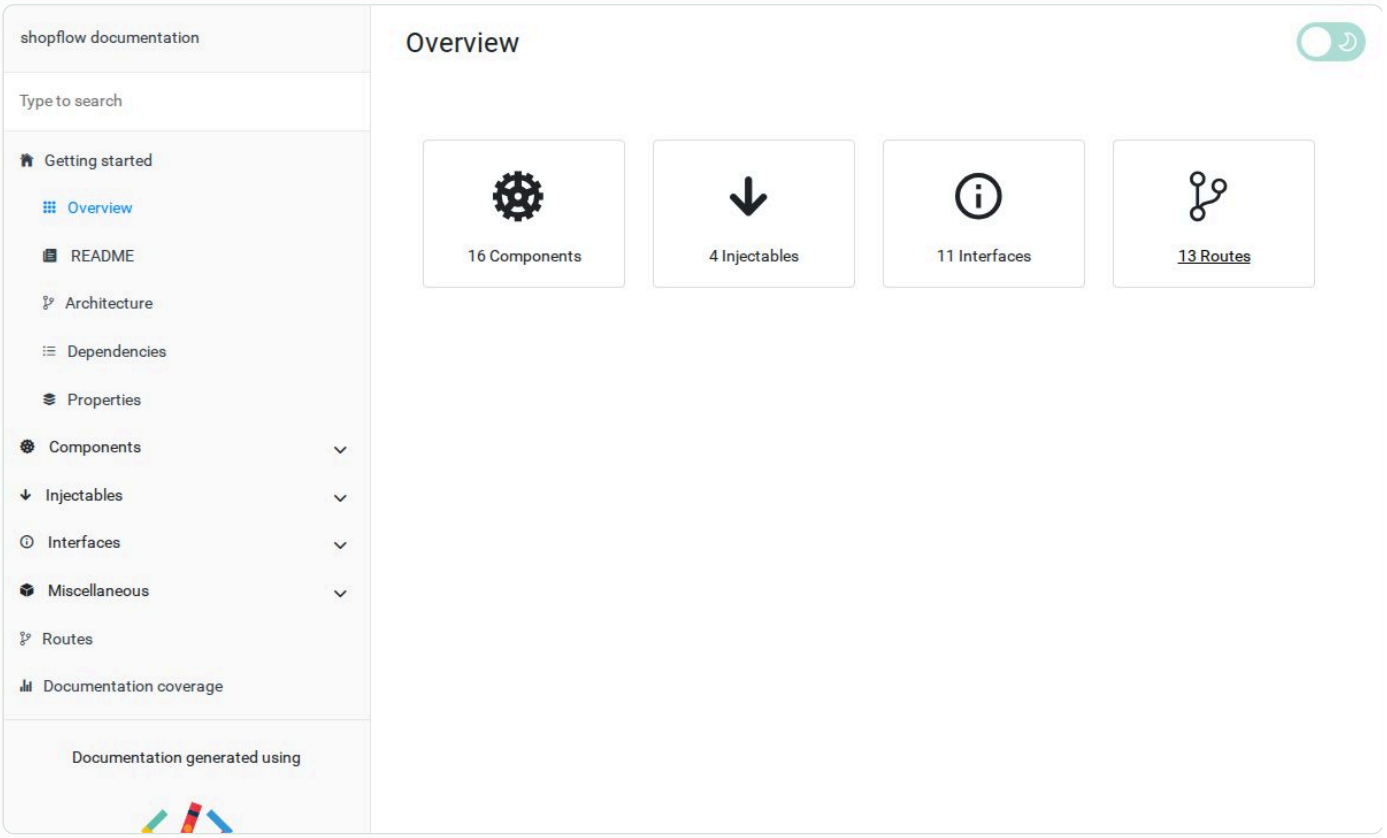
tsconfig.doc.json `{`

```
{
  "extends": "./tsconfig.json",
  "include": ["src/**/*.ts", "libs/**/*.ts"],
  "exclude": ["**/*.spec.ts"]
}
```

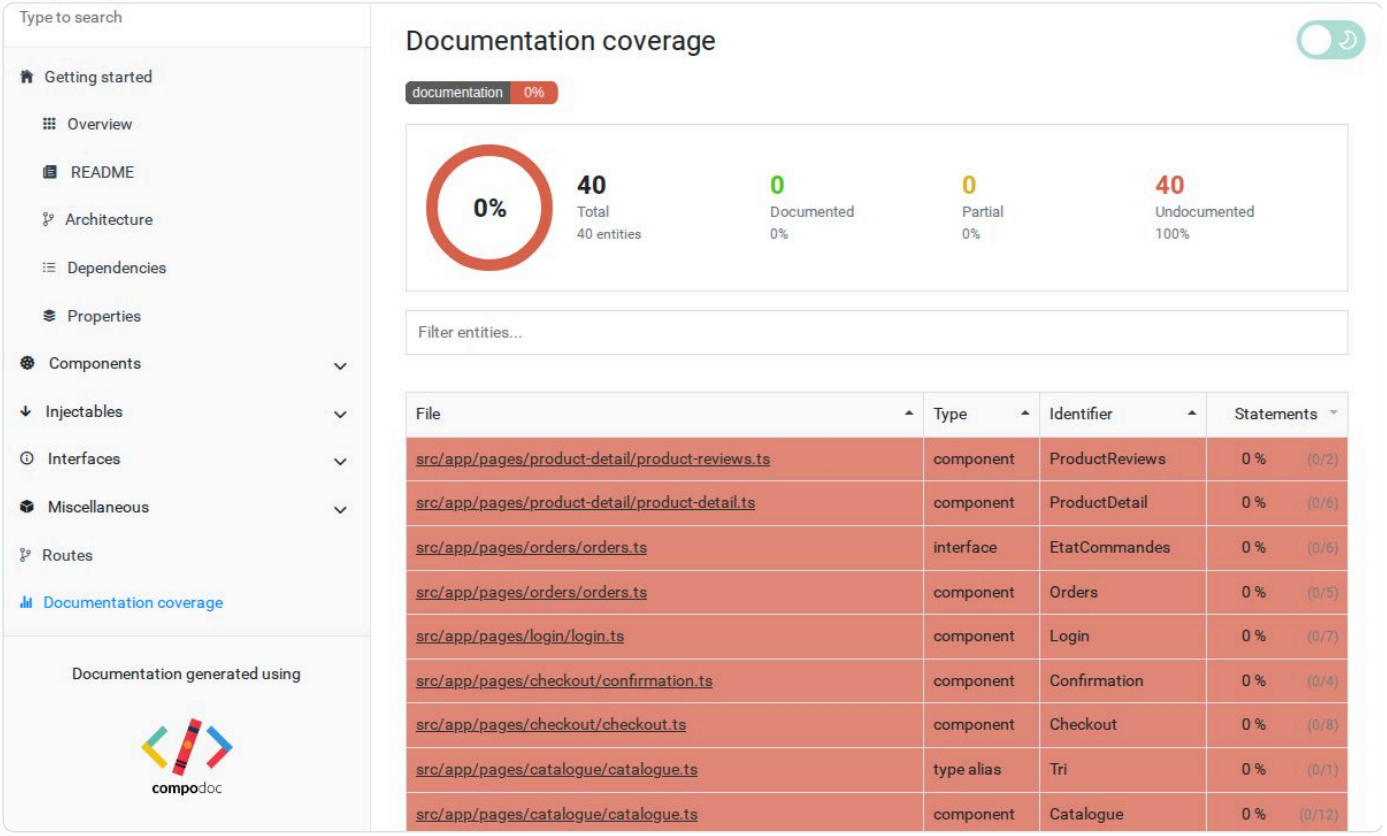
Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) `npx nx run shopflow:doc`

```
npx nx run shopflow:doc
```

4. Le résultat



La page d'accueil : 16 composants, 4 services, 11 interfaces, 13 routes, sans avoir écrit une ligne de documentation.



Documentation coverage : 0 % des 40 éléments ont une description.

10 h 20 – 10 h 40

Ne pas faire travailler l’affichage pour rien

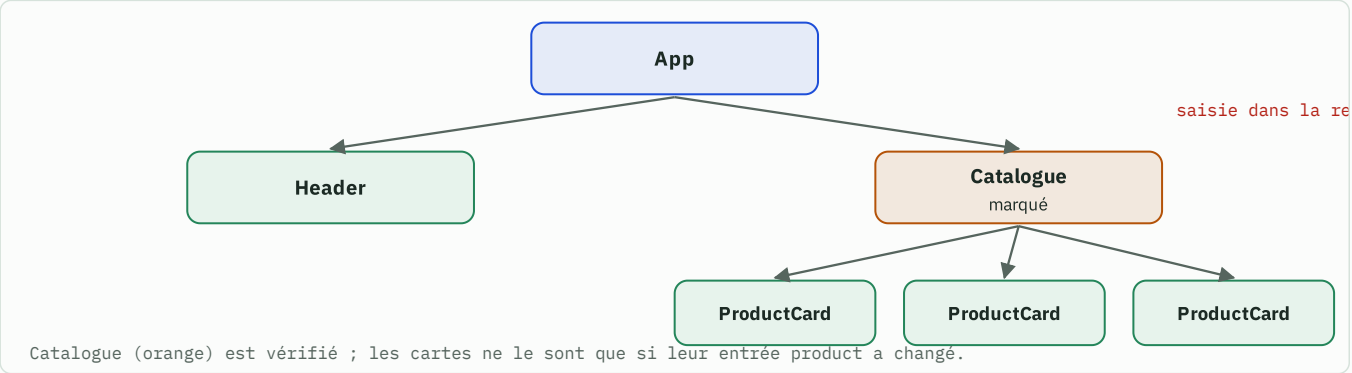
Module 1, « Data binding, mode manuel : ChangeDetectorRef ».

1. La notion

Objectif : éviter que l’affichage refasse des calculs inutiles, et savoir le prévenir quand il ne voit pas un changement.

Pourquoi : une méthode appelée dans un template est réexécutée à chaque vérification de la page, même quand rien n’a changé.

Dans la réalité : sur une liste de 194 produits, taper « phone » dans la recherche recalcule 88 fois le prix remisé. Sur un téléphone d’entrée de gamme, cela se sent.



Une frappe dans la recherche marque Catalogue et ses parents ; Header n’est pas vérifié.

Trois outils : `track` par identifiant dans `@for` (Angular ne recrée pas les éléments inchangés) ; un **pipe pur** à la place d’une méthode (recalculé seulement si sa valeur d’entrée change) ; `ChangeDetectorRef` pour prévenir Angular quand une valeur change sans qu’il le voie.

2. La démonstration

Le calcul du prix est une méthode de la carte, appelée dans le template : 88 appels pendant la saisie de « phone » (compteur affiché dans la console).

3. La correction

src/app/pages/catalogue/catalogue.html (extrait) (extrait) @for (p of

```
@for (p of produitsAffiches(page.products); track p.id) {
```

libs/shared/ui/src/lib/prix-remise/prix-remise-pipe.ts

```ts
import { Pipe, PipeTransform } from '@angular/core';
import { Product, prixRemise } from '@shopflow/models';

/** Prix d'un produit après remise. */
@Pipe({ name: 'prixRemise' })
export class PrixRemisePipe implements PipeTransform {
  transform(produit: Product): number {
    return prixRemise(produit);
  }
}
```

libs/shared/ui/src/lib/product-card/product-card.html

```html
@let p = product();
<article class="carte">
  <a [routerLink]="['/catalogue', p.id]">
    <img class="carte-image" [ngSrc]="p.thumbnail" [alt]="p.title" width="300" height="300"
[priority]="prioriteInitiale" />
  </a>
  <div class="carte-corps">
    <p class="carte-categorie">{{ p.category }}</p>
    <h2 class="carte-titre"><a [routerLink]="['/catalogue', p.id]">{{ p.title }}</a></h2>
    <sf-note [valeur]="p.rating" />
    <p class="carte-prix">
      {{ p | prixRemise | currency: 'EUR' }}
      @if (p.discountPercentage > 0) {
        <s class="prix-barre">{{ p.price | currency: 'EUR' }}</s>
      }
    </p>
    <button class="bouton" type="button" (click)="ajouter()">Ajouter au panier</button>
  </div>
</article>
```

Quand Angular ne voit pas le changement : le tableau de bord peut figer l'affichage des commandes, puis le relancer à la demande.

src/app/admin/live-orders.ts

```ts
import { ChangeDetectorRef, Component, inject, OnInit } from '@angular/core';
import { AsyncPipe, CurrencyPipe, DatePipe } from '@angular/common';
import { BehaviorSubject } from 'rxjs';
import { Order } from '@shopflow/models';
import { OrderService } from '@shopflow/data-access';

@Component({
  selector: 'app-live-orders',
  imports: [AsyncPipe, CurrencyPipe, DatePipe],
  templateUrl: './live-orders.html',
})
export class LiveOrders implements OnInit {
  private readonly orderService = inject(OrderService);
  private readonly cdr = inject(ChangeDetectorRef);
```

```
  protected readonly commandes$ = new BehaviorSubject<Order[]>([]);
  protected readonly miseAJour$ = new BehaviorSubject<Date | null>(null);
  protected figé = false;
  private position = 0;

  ngOnInit(): void {
    this.charger();
    setInterval(() => this.charger(), 5000);
  }

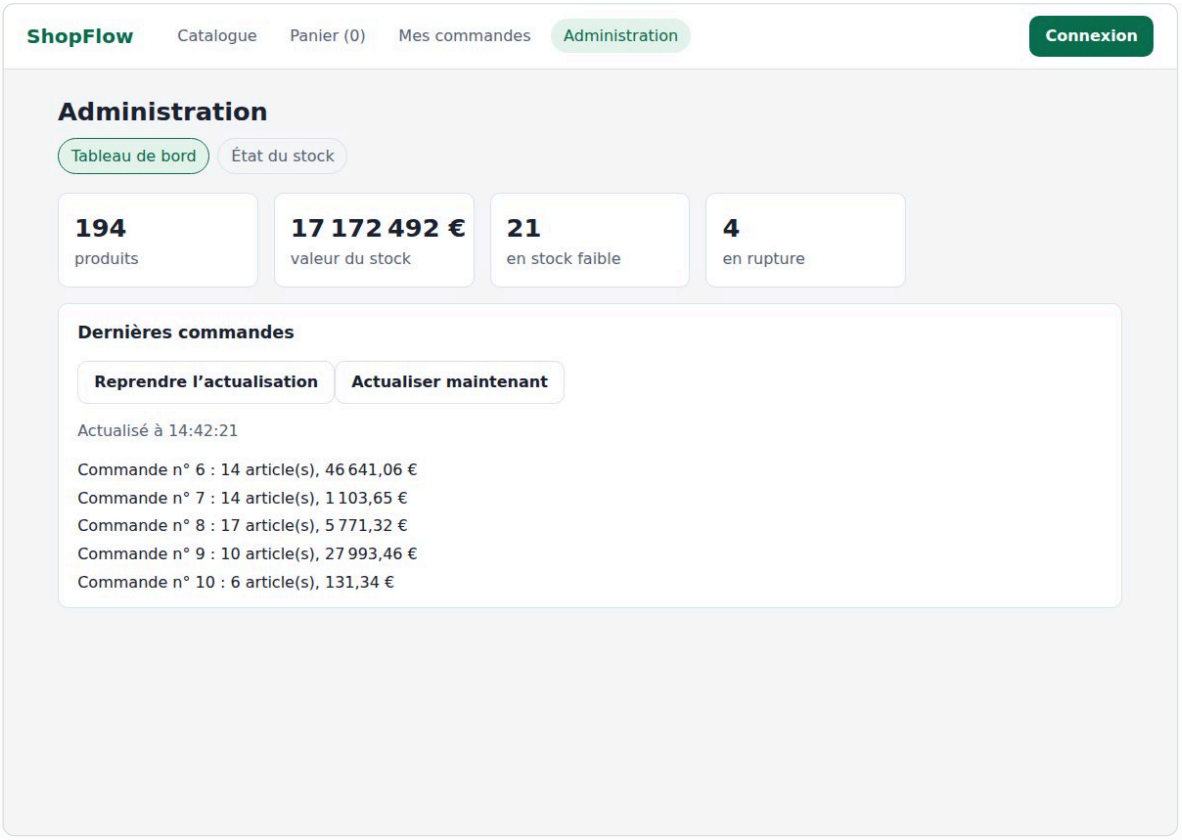
  protected basculer(): void {
    this.figé = !this.figé;
    if (this.figé) {
      this.cdr.detach();
      this.cdr.detectChanges();
    } else {
      this.cdr.reattach();
    }
  }

  protected actualiser(): void {
    this.cdr.detectChanges();
  }

  private charger(): void {
    // Simulation : l'API de démonstration ne reçoit pas de nouvelles commandes, on fait défiler ses paniers
    this.position = (this.position + 5) % 200;
    this.orderService.latest(this.position).subscribe((page) => {
      this.commandes$.next(page.carts);
      this.miseAJour$.next(new Date());
    });
  }
}
```

4. Le résultat

Mesure	Avant	Après
Calculs du prix pendant la saisie de « phone »	88	0
Heure affichée après « Figer »	—	inchangée, jusqu'à « Actualiser maintenant »



Affichage figé : l'heure reste à 14:42:21.

10 h 40 – 10 h 55 : pause

10 h 55 – 11 h 15

Zone.js : ce qui a changé

Module 2, « ZoneJS : le concept ». Explication seulement : Angular 22 n'utilise plus Zone.js ; on le fera avec les signaux (cet après-midi).

Avant (jusqu'à Angular 20) : la bibliothèque **Zone.js** surveillait tout ce qui se passait dans la page (clics, minuteries, réponses du serveur). Après chacun de ces événements, Angular revérifiait **toute** la page, au cas où quelque chose aurait changé. Simple pour le développeur, mais coûteux : on vérifiait des centaines de composants pour rien.

Aujourd'hui (Angular 21 et suivants, donc ShopFlow en Angular 22) : il n'y a plus de Zone.js. Angular ne redessine un composant que lorsqu'on le lui signale : un événement dans son template, un pipe `async`, ou **un signal qui change**. On le fera avec les signaux, cet après-midi : c'est là qu'on verra comment ils prennent le relais.

	Avec Zone.js (avant)	**Sans Zone.js (aujourd'hui)**
Qui déclenche l'affichage	Tout événement de la page, automatiquement	Un signal qui change, un événement du template, un pipe `async`
Ce qui est revérifié	Toute la page	Seulement les composants concernés
Une valeur changée dans un `setInterval`	S'affiche toute seule	Ne s'affiche pas, sauf si c'est un signal

> **Dans le code existant**
>
> Une application qui utilise encore Zone.js se reconnaît à `zone.js` dans les `polyfills` d'`angular.json`, à `provideZoneChangeDetection()` dans `app.config.ts`, ou à l'injection de `NgZone`. Pour la migrer, on remplace les valeurs affichées par des signaux, puis on retire Zone.js.

11 h 15 – 11 h 40

RxJS : partager une requête

Module 2, « Choisir RxJS, utilisation des observables ».

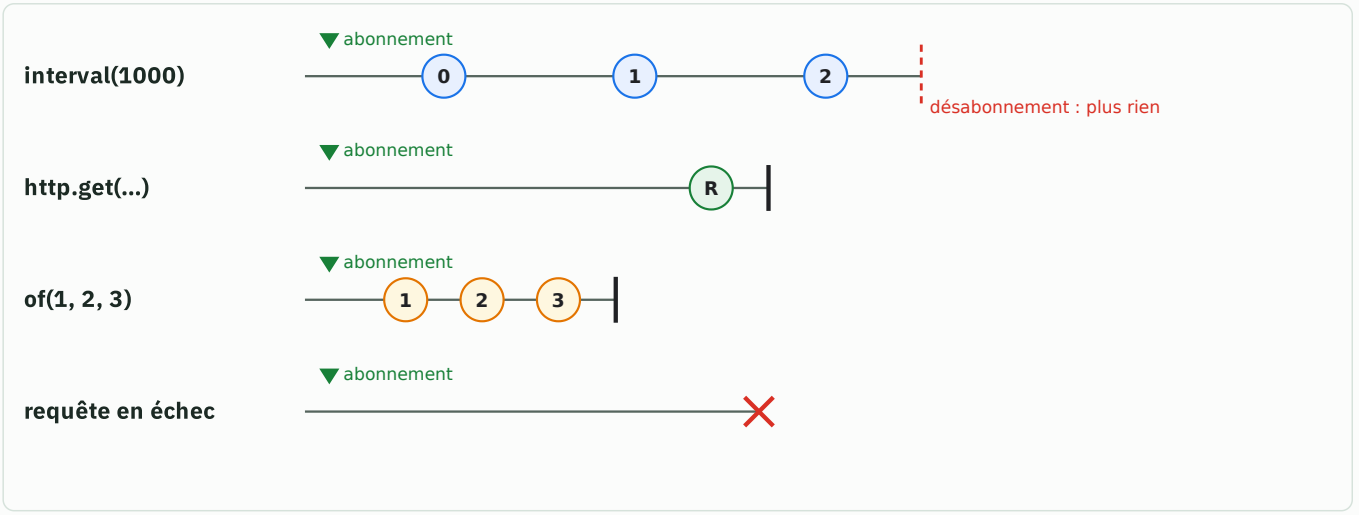
1. La notion

Objectif : que le catalogue ne soit téléchargé qu'une fois, même si trois pages l'utilisent.

Pourquoi : chaque abonnement à un appel HTTP relance la requête : le catalogue, le tableau de bord et le stock téléchargent chacun les 194 produits.

Dans la réalité : trois téléchargements identiques, c'est trois fois plus de charge pour le serveur et trois fois plus de données pour le client.

> **Lire un diagramme en billes**
>
> Chaque ligne est un **flux**, la flèche est **le temps qui passe**, de gauche à droite.
>
> - un **cercle** : une valeur émise ;
> - une **barre verticale noire** : le flux est terminé (`complete`) ;
> - une **croix rouge** : une erreur ;
> - le **triangle vert** : le moment où l'on s'abonne ;
> - le **trait rouge pointillé** : le désabonnement (le flux s'arrête pour cet abonné) ;
> - une **boîte grise** : un opérateur ; la ligne au-dessus entre dans l'opérateur, la ligne en dessous en sort.



Quatre flux : une minuterie qui ne finit jamais (on l'arrête en se désabonnant), une requête HTTP (une réponse puis la fin), trois valeurs immédiates, une erreur.



R = une requête vers /products?limit=0.

> **Voir les billes bouger**
>
> Des sites où l'on déplace soi-même les billes pour voir réagir un opérateur :
>
> - **rxjsmarbles.dev** : réécriture à jour de RxMarbles, un lien par opérateur (par exemple `https://rxjsmarbles.dev/debounceTime`) ;
> - **rxmarbles.com** : le site historique, toujours en ligne (par exemple `https://rxmarbles.com/#switchMap`) ;
> - **rxviz.com** : on colle son propre code RxJS et le site dessine le flux en direct ;
> - **rxjs.dev**, la documentation officielle : chaque opérateur y a son diagramme ;
> - **learnrxjs.io** : chaque opérateur expliqué avec des exemples.

2. La démonstration

Onglet Réseau, filtre `limit=0` : ouvrir le catalogue, l'administration, puis l'état du stock : 3 requêtes identiques.

3. La correction

libs/shared/data-access/src/lib/services/product-service.ts — import { HttpClient } from

```ts
import { HttpClient } from '@angular/common/http';
import { inject, Service } from '@angular/core';
import { Observable, shareReplay } from 'rxjs';
import { Category, Product, ProductPage } from '@shopflow/models';
import { API_URL } from './api';

/**
 * Accès au catalogue de l'API DummyJSON.
 *
 * Le catalogue complet est demandé une seule fois, puis partagé entre tous les composants.
 */
@Service()
export class ProductService {
  private readonly http = inject(HttpClient);

  private readonly catalogue$ = this.http.get<ProductPage>(`${API_URL}/products?limit=0`).pipe(
    shareReplay({ bufferSize: 1, refCount: false }),
  );
```

```
  /**
   * Tout le catalogue (194 produits), demandé à l'API au premier appel seulement.
   * @returns La page de résultats : produits et nombre total
   */
  getAll(): Observable<ProductPage> {
    return this.catalogue$;
  }

  /**
   * Un produit, avec ses avis.
   * @param id Identifiant du produit
   */
  getById(id: number): Observable<Product> {
    return this.http.get<Product>(`${API_URL}/products/${id}`);
  }

  /** Les catégories du catalogue, pour le filtre. */
  getCategories(): Observable<Category[]> {
    return this.http.get<Category[]>(`${API_URL}/products/categories`);
  }
}
```

En deux mots

`shareReplay`, c'est **enregistrer le film une fois** et le repasser à chaque nouveau spectateur, au lieu de relancer le tournage.

1. La première page qui s'abonne déclenche la requête.
2. La réponse est gardée.
3. Les pages suivantes reçoivent la réponse gardée, sans nouvelle requête.

4. Le résultat

Mesure	Avant	Après
Requêtes `products?limit=0` (catalogue, tableau de bord, stock)	3	1

11 h 40 – 12 h 10

RxJS : une recherche qui attend la fin de la frappe

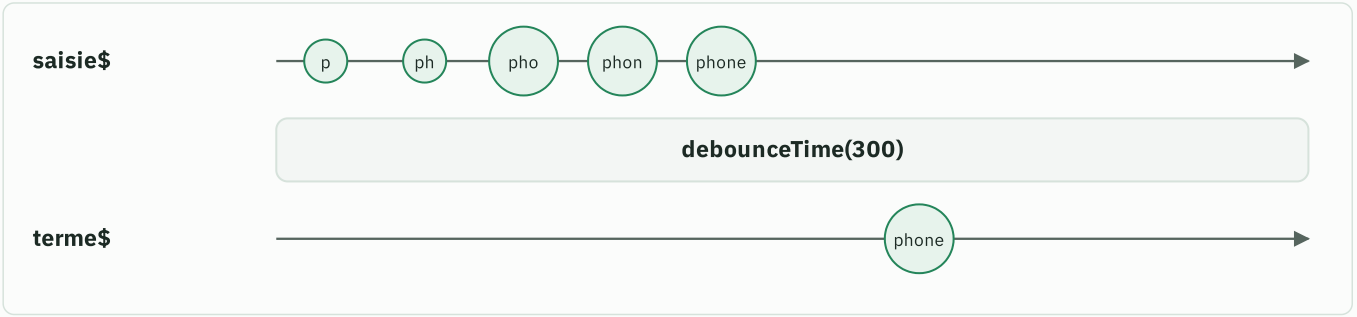
Module 2, « Choisir RxJS, utilisation des observables ».

1. La notion

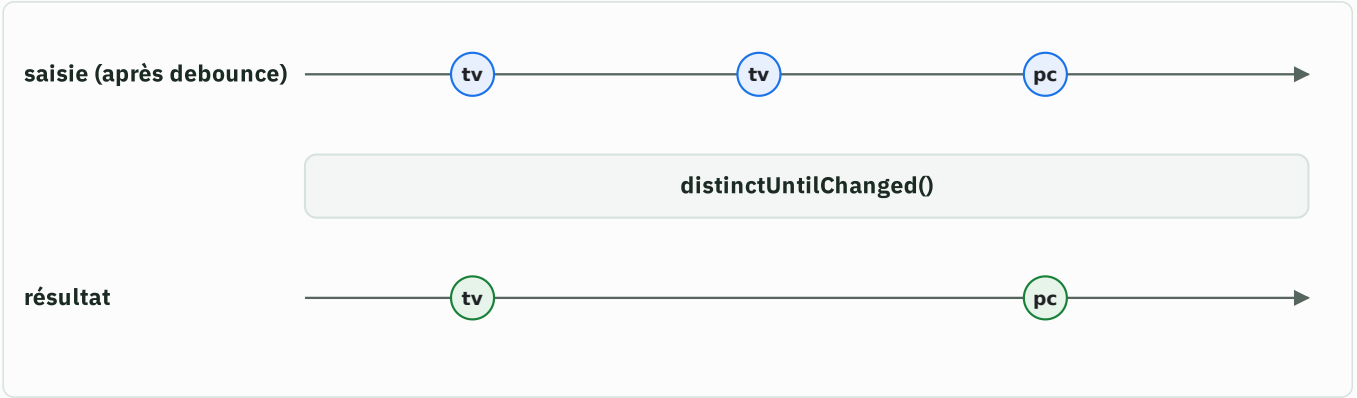
Objectif : que la liste ne soit recalculée qu'une fois la saisie terminée.

Pourquoi : chaque lettre tapée relance le filtrage de 194 produits.

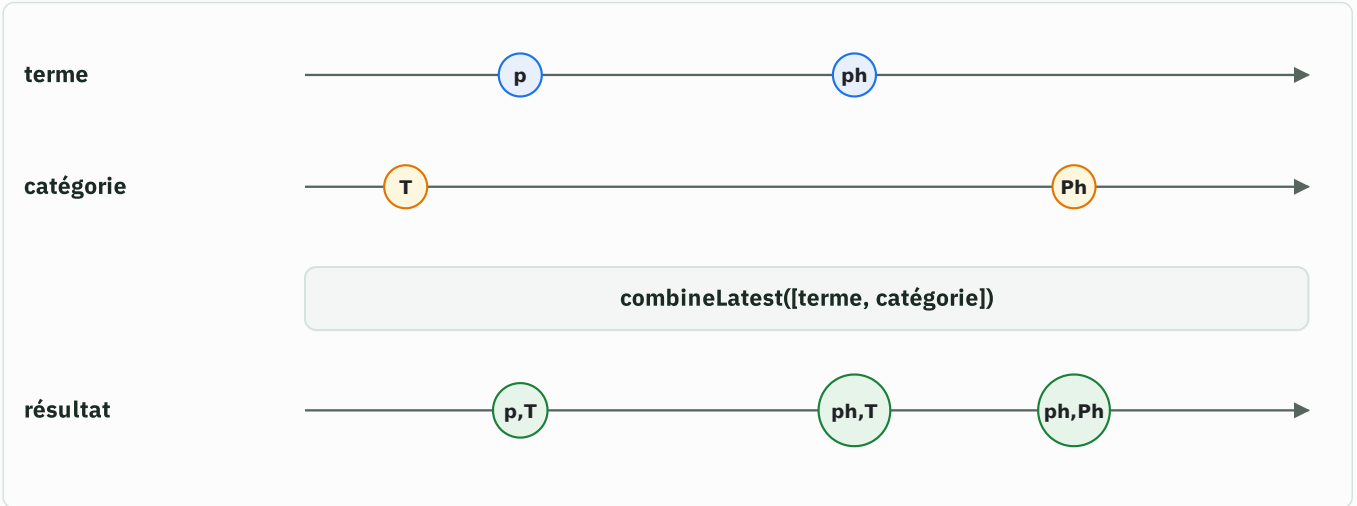
Dans la réalité : sur un site qui interroge le serveur à chaque lettre, taper « phone » envoie cinq requêtes, et une réponse lente peut afficher les résultats d'une recherche dépassée.



debounceTime ne laisse passer une valeur que si 300 ms s'écoulent sans nouvelle frappe.



distinctUntilChanged ignore une valeur identique à la précédente : « tv » retapé ne relance rien.



combineLatest émet la dernière valeur de chaque flux dès que l'un d'eux change : la liste est recalculée avec le terme ET la catégorie.

2. La démonstration

Taper « phone » dans la recherche : la liste est recalculée 5 fois (compteur dans la console).

3. La correction

src/app/pages/catalogue/catalogue.ts

```ts
import { Component, inject } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { BehaviorSubject, combineLatest, debounceTime, distinctUntilChanged, map, startWith, Subject } from 'rxjs';
import { Product } from '@shopflow/models';
import { CartService, ProductService } from '@shopflow/data-access';
import { ProductCard } from '@shopflow/ui';

type Tri = 'nom' | 'prix-croissant' | 'prix-decroissant';

/** Filtre et trie les produits : fonction pure, sans état, facile à tester. */
function filtrerEtTrier(produits: Product[], terme: string, categorie: string, tri: Tri): Product[] {
  const filtres = produits.filter((p) => p.title.toLowerCase().includes(terme) && (!categorie || p.category === categorie));
  return [...filtres].sort((a, b) =>
    tri === 'nom' ? a.title.localeCompare(b.title) : tri === 'prix-croissant' ? a.price - b.price : b.price - a.price,
  );
}

@Component({
  selector: 'app-catalogue',
  imports: [AsyncPipe, ProductCard],
  templateUrl: './catalogue.html',
})
export class Catalogue {
  private readonly productService = inject(ProductService);
  protected readonly panier = inject(CartService);
  protected readonly categories$ = this.productService.getCategories();

  private readonly saisie$ = new Subject<string>();
  private readonly categorie$ = new BehaviorSubject('');
  private readonly tri$ = new BehaviorSubject<Tri>('nom');

  private readonly terme$ = this.saisie$.pipe(
    debounceTime(300),
    map((t) => t.trim().toLowerCase()),
    distinctUntilChanged(),
    startWith(''),
  );

  protected readonly vue$ = combineLatest([this.productService.getAll(), this.terme$, this.categorie$, this.tri$]).pipe(
    map(([page, terme, categorie, tri]) => ({ total: page.total, produits: filtrerEtTrier(page.products, terme, categorie, tri) })),
```

```
  );

  protected onRecherche(event: Event): void {
    this.saisie$.next((event.target as HTMLInputElement).value);
  }

  protected onCategorie(event: Event): void {
    this.categorie$.next((event.target as HTMLSelectElement).value);
  }

  protected onTri(event: Event): void {
    this.tri$.next((event.target as HTMLSelectElement).value as Tri);
  }
}
```

En deux mots

La recherche, c'est **attendre que la personne ait fini de parler** avant de répondre.

1. `debounceTime(300)` attend 300 ms sans nouvelle frappe.
2. `distinctUntilChanged()` ignore un terme identique au précédent.
3. `combineLatest` recalcule la liste quand le terme ou la catégorie change.

src/app/pages/catalogue/catalogue.html <h1>Catalogue</h1>

```
<h1>Catalogue</h1>
@if (vue$ | async; as vue) {
  <p class="sous-titre">{{ vue.total }} produits</p>
  <div class="barre-outils">
    <input class="champ recherche" type="search" placeholder="Rechercher un produit…"
(input)="onRecherche($event)" />
    <select class="champ" (change)="onCategorie($event)">
      <option value="">Toutes les catégories</option>
      @for (c of categories$ | async; track c.slug) {
        <option [value]="c.slug">{{ c.name }}</option>
      }
    </select>
    <select class="champ" (change)="onTri($event)">
      <option value="nom">Trier par nom</option>
      <option value="prix-croissant">Prix croissant</option>
      <option value="prix-decroissant">Prix décroissant</option>
    </select>
  </div>
  <div class="grille">
    @for (p of vue.produits; track p.id) {
      <sf-product-card [product]="p" [prioritaire]="$index < 4" (ajout)="panier.add($event)" />
    } @empty {
      <p class="info">Aucun produit ne correspond à votre recherche.</p>
    }
  </div>
} @else {
```

```
  <p class="info">Chargement du catalogue…</p>
}
```

4. Le résultat

Mesure	Avant	Après
Calculs de la liste pendant la saisie de « phone »	5	1

12 h 10 – 12 h 30

RxJS : enchaîner deux requêtes sans les imbriquer

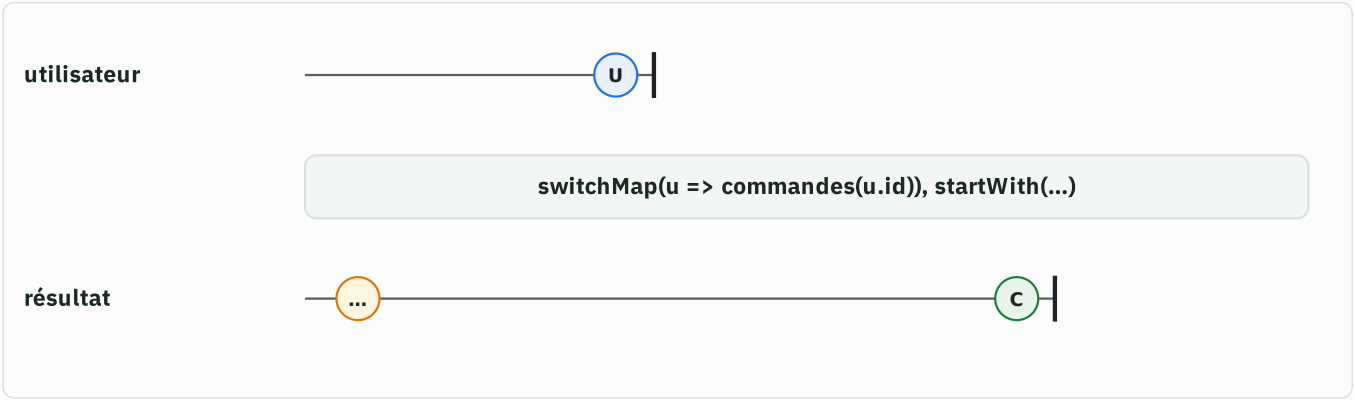
Module 2, « Choisir RxJS, utilisation des observables ».

1. La notion

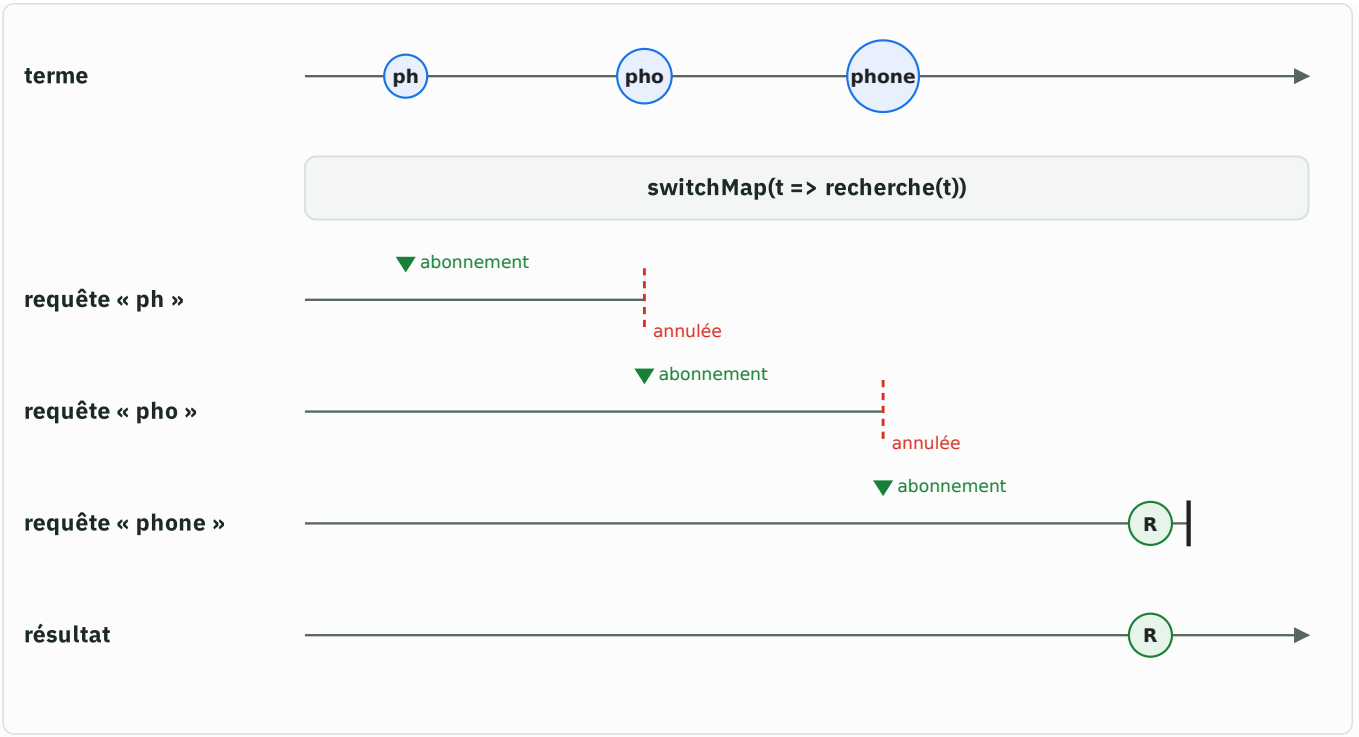
Objectif : charger l'utilisateur puis ses commandes en un seul flux lisible.

Pourquoi : Mes commandes imbrique un `subscribe` dans un autre : le code est difficile à lire, les erreurs et le chargement sont mal gérés, et rien n'annule la seconde requête si l'on quitte la page.

Dans la réalité : ce motif « une requête, puis une autre qui dépend de la première » est partout : un client puis ses commandes, une commande puis ses produits.



L'utilisateur arrive, puis ses commandes ; startWith affiche « chargement » (…) en attendant.



switchMap lance une requête par terme et annule la précédente dès qu'un nouveau terme arrive : seule la réponse de « phone » est affichée.

2. La démonstration

src/app/pages/orders/orders.ts (avant, extrait) (extrait) — import { Order } from

```ts
import { Order } from '../../models/order';
import { User } from '../../models/user';
import { AuthService } from '../../services/auth-service';
import { OrderService } from '../../services/order-service';
```

3. La correction

src/app/pages/orders/orders.ts — import { Component, inject } from

```ts
import { Component, inject } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { HttpErrorResponse } from '@angular/common/http';
import { catchError, map, Observable, of, startWith, switchMap } from 'rxjs';
import { Order, User } from '@shopflow/models';
import { AuthService, OrderService } from '@shopflow/data-access';

interface EtatCommandes {
  chargement: boolean;
  erreur?: string;
  utilisateur?: User;
  commandes: Order[];
  totalDepense: number;
}

@Component({
  selector: 'app-orders',
  imports: [AsyncPipe, CurrencyPipe],
```

```
  templateUrl: './orders.html',
})
export class Orders {
  private readonly auth = inject(AuthService);
  private readonly orderService = inject(OrderService);

  protected readonly etat$: Observable<EtatCommandes> = this.auth.me().pipe(
    switchMap((utilisateur) =>
      this.orderService.byUser(utilisateur.id).pipe(
        map((page) => ({
          chargement: false,
          utilisateur,
          commandes: page.carts,
          totalDepense: page.carts.reduce((s, c) => s + c.discountedTotal, 0),
        })),
      ),
    ),
    catchError((erreur: HttpErrorResponse) =>
      of({
        chargement: false,
        erreur: erreur.status === 401 ? 'Connectez-vous pour voir vos commandes.' : 'Impossible de charger vos commandes.',
        commandes: [],
        totalDepense: 0,
      }),
    ),
    startWith({ chargement: true, commandes: [], totalDepense: 0 }),
  );
}
```

En deux mots

`switchMap`, c'est **passer le relais** : la première requête donne l'utilisateur, `switchMap` lance avec lui la requête des commandes, et abandonne la précédente si une nouvelle arrive.

1. Un seul flux, lu de haut en bas.
2. Le pipe `async` s'abonne et se désabonne tout seul.
3. `startWith` affiche « chargement » pendant l'attente.

4. Le résultat

Même affichage qu'avant, un seul abonnement, et plus aucune requête après avoir quitté la page.

12 h 30 – 13 h 30 : déjeuner

13 h 30 – 13 h 50

RxJS : arrêter une minuterie (fuite de mémoire)

Module 2, « Choisir RxJS, utilisation des observables ».

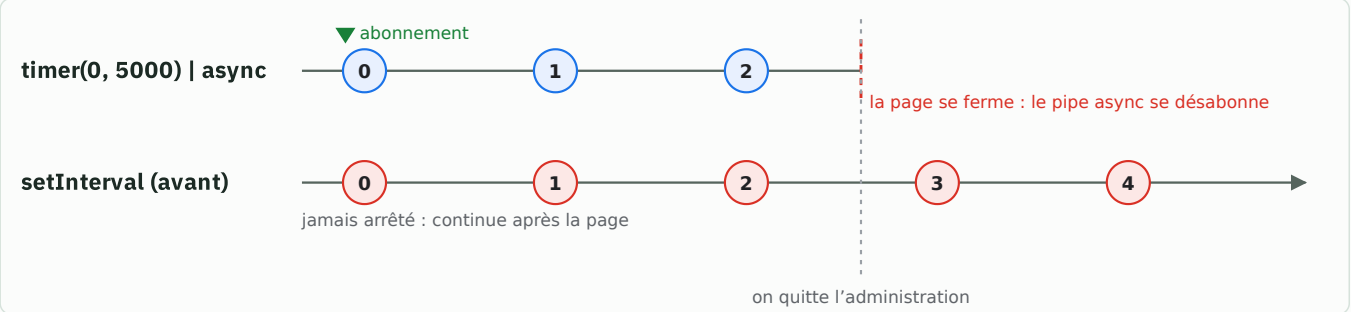
1. La notion

Objectif : que la mise à jour automatique des commandes s'arrête quand on quitte l'administration.

Pourquoi : la minuterie (`setInterval`) n'est jamais arrêtée : elle continue après la page, et chaque visite en ajoute une.

Dans la réalité : une application ouverte toute la journée par un vendeur accumule les minuteries : le serveur reçoit de plus en plus de requêtes, et le navigateur ralentit.

> **En deux mots**
>
> Une minuterie jamais arrêtée, c'est une **lumière laissée allumée en quittant la pièce** : on ne la voit plus, mais elle consomme, et chaque passage en allume une de plus.



Avec le pipe async, la minuterie s'arrête quand la page se ferme ; le setInterval de départ continue, et chaque visite en ajoute un.

2. La démonstration

Onglet Réseau, filtre `carts` : ouvrir l'administration, revenir au catalogue, attendre : une requête toutes les 5 secondes ; après une deuxième visite, deux.

3. La correction

src/app/admin/live-orders.ts

```
import { ChangeDetectorRef, Component, inject } from '@angular/core';
import { AsyncPipe, CurrencyPipe, DatePipe } from '@angular/common';
import { map, switchMap, timer } from 'rxjs';
import { OrderService } from '@shopflow/data-access';

@Component({
  selector: 'app-live-orders',
  imports: [AsyncPipe, CurrencyPipe, DatePipe],
  templateUrl: './live-orders.html',
})
```

```
export class LiveOrders {
  private readonly orderService = inject(OrderService);
  private readonly cdr = inject(ChangeDetectorRef);
  protected figé = false;

  protected readonly flux$ = timer(0, 5000).pipe(
    map((n) => ((n + 1) * 5) % 200),
    switchMap((position) => this.orderService.latest(position)),
    map((page) => ({ commandes: page.carts, date: new Date() })),
  );

  protected basculer(): void {
    this.figé = !this.figé;
    if (this.figé) {
      this.cdr.detach();
      this.cdr.detectChanges();
    } else {
      this.cdr.reattach();
    }
  }

  protected actualiser(): void {
    this.cdr.detectChanges();
  }
}
```

En deux mots

`timer(0, 5000)` émet tout de suite, puis toutes les 5 secondes ; le pipe `async` s'abonne à l'affichage et **se désabonne à la sortie de la page** : la minuterie s'arrête avec elle.

src/app/admin/live-orders.html `<section class="panneau">`

```
<section class="panneau">
  <h2>Dernières commandes</h2>
  <p>
    <button class="bouton bouton-secondaire" type="button" (click)="basculer()">{{ figé ? 'Reprendre l'actualisation' : 'Figer l'affichage' }}</button>
    @if (figé) {
      <button class="bouton bouton-secondaire" type="button" (click)="actualiser()">Actualiser maintenant</button>
    }
  </p>
  @if (flux$ | async; as flux) {
    <p class="info">Actualisé à {{ flux.date | date: 'HH:mm:ss' }}</p>
    <ul class="liste-simple">
      @for (c of flux.commandes; track c.id) {
        <li>Commande n° {{ c.id }} : {{ c.totalQuantity }} article(s), {{ c.discountedTotal | currency: 'EUR' }}</li>
      }
    </ul>
```

```
  }
</section>
```

4. Le résultat

Mesure	Avant	Après
Requêtes `carts` dans les 20 s après la sortie de l'administration	4	0

13 h 50 – 14 h 10

Réserver des pages aux personnes connectées

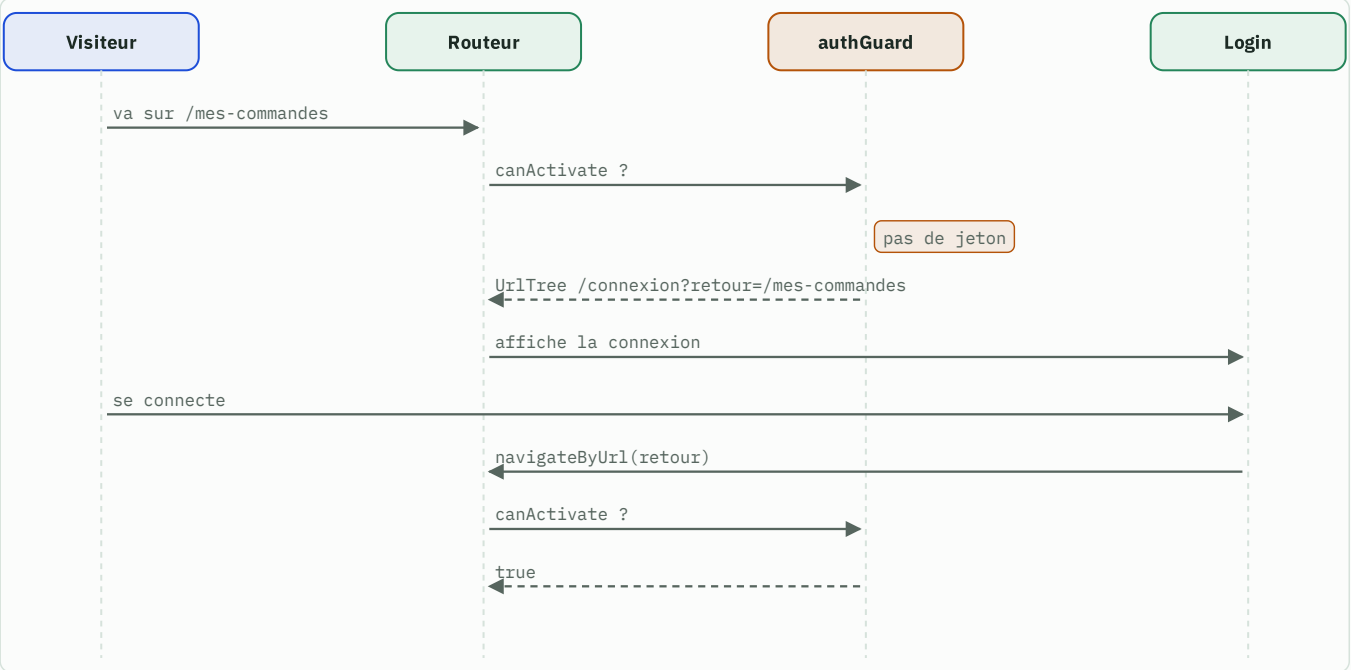
Module 2, « Route guards : CanActivate, CanActivateChild ».

1. La notion

Objectif : qu'on ne puisse ouvrir Mes commandes et l'administration qu'une fois connecté.

Pourquoi : aujourd'hui, ces pages s'ouvrent sans session ; il suffit de taper l'adresse.

Dans la réalité : un espace client ou un back-office accessible sans connexion, c'est une fuite de données. Le guard est la première barrière côté navigateur (le serveur, lui, doit aussi refuser).



Le guard redirige vers la connexion en mémorisant la page demandée ; après connexion, on y revient.

2. La démonstration

Sans être connecté, ouvrir `http://localhost:4200/mes-commandes` : la page s'ouvre (et échoue à charger les commandes).

3. La correction

src/app/guards/auth-guard.ts — `import { inject } from '@angular/core';`

```ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '@shopflow/data-access';

/** Laisse passer un utilisateur connecté ; sinon, renvoie vers la connexion en mémorisant la page demandée. */
export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.token
    ? true
    : router.createUrlTree(['/connexion'], { queryParams: { retour: state.url } });
};
```

src/app/app.routes.ts — `import { Routes } from '@angular/router';`

```ts
import { Routes } from '@angular/router';
import { Catalogue } from './pages/catalogue/catalogue';
import { authGuard } from './guards/auth-guard';
import { productResolver } from './pages/product-detail/product-resolver';

export const routes: Routes = [
  { path: '', redirectTo: 'catalogue', pathMatch: 'full' },
  { path: 'catalogue', component: Catalogue },
  {
    path: 'catalogue/:id',
    loadComponent: () => import('./pages/product-detail/product-detail').then((m) => m.ProductDetail),
    resolve: { produit: productResolver },
  },
  { path: 'panier', loadComponent: () => import('./pages/cart/cart').then((m) => m.Cart) },
  { path: 'commande', loadComponent: () => import('./pages/checkout/checkout').then((m) => m.Checkout) },
  { path: 'commande/confirmation', loadComponent: () => import('./pages/checkout/confirmation').then((m) =>
m.Confirmation) },
  { path: 'connexion', loadComponent: () => import('./pages/login/login').then((m) => m.Login) },
  {
    path: 'mes-commandes',
    canActivate: [authGuard],
    loadComponent: () => import('./pages/orders/orders').then((m) => m.Orders),
  },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes').then((m) => m.adminRoutes) },
  { path: '**', redirectTo: 'catalogue' },
];
```

Toute l'administration d'un coup, avec une seule déclaration sur la route parente :

src/app/guards/admin-guard.ts — `import { inject } from '@angular/core';`

```ts
import { inject } from '@angular/core';
import { CanActivateChildFn, Router } from '@angular/router';
import { catchError, map, of } from 'rxjs';
```

```ts
import { AuthService } from '@shopflow/data-access';

/** Réserve les pages enfants de l'administration au rôle « admin ». */
export const adminGuard: CanActivateChildFn = (_childRoute, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const versConnexion = router.createUrlTree(['/connexion'], { queryParams: { retour: state.url } });
  if (!auth.token) {
    return versConnexion;
  }
  return auth.me().pipe(
    map((utilisateur) => (utilisateur.role === 'admin' ? true : router.createUrlTree(['/catalogue']))),
    catchError(() => of(versConnexion)),
  );
};
```

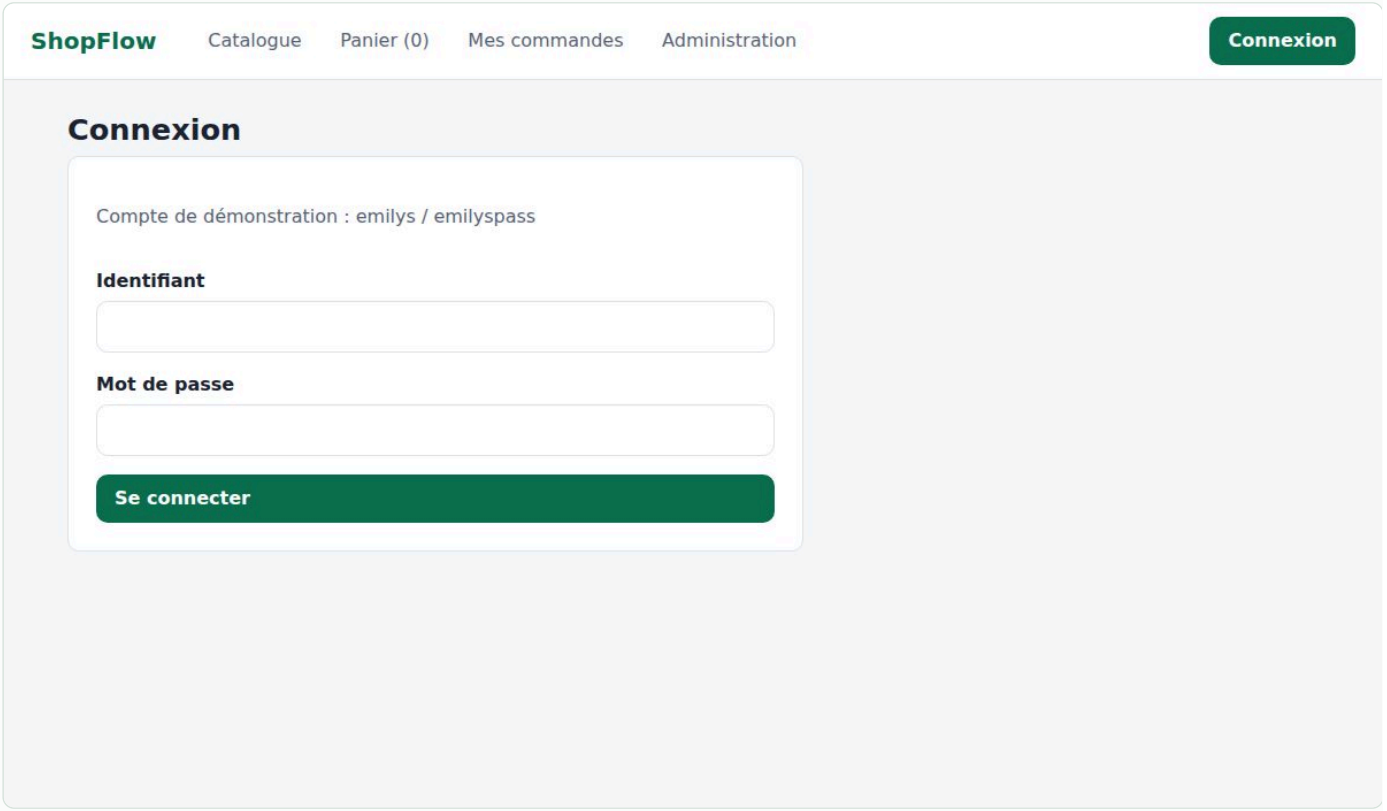
src/app/admin/admin.routes.ts

```ts
import { Routes } from '@angular/router';
```

```ts
import { Routes } from '@angular/router';
import { adminGuard } from '../guards/admin-guard';
import { AdminLayout } from './admin-layout';
import { AdminDashboard } from './admin-dashboard';
import { StockTable } from './stock-table';

export const adminRoutes: Routes = [
  {
    path: '',
    component: AdminLayout,
    canActivateChild: [adminGuard],
    children: [
      { path: '', component: AdminDashboard },
      { path: 'stock', component: StockTable },
    ],
  },
];
```

4. Le résultat



Sans session, /mes-commandes mène à /connexion?retour=/mes-commandes.

Sans session, `/admin` mène aussi à `/connexion?retour=/admin` ; après connexion, on revient à la page demandée.

14 h 10 – 14 h 25

Charger les données avant d'afficher la page

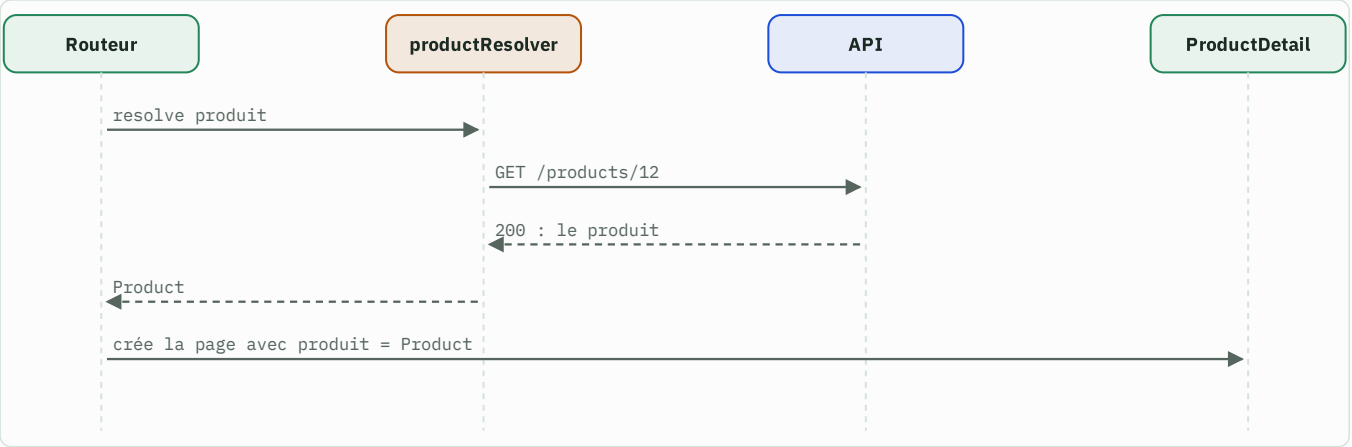
Module 2, « Pre-fetching avec resolve ».

1. La notion

Objectif : que la fiche produit s'affiche directement remplie.

Pourquoi : la fiche s'ouvre vide, puis se remplit quand la requête répond : la page « clignote ».

Dans la réalité : sur une connexion lente, le client voit une page vide ou à moitié remplie ; avec un resolver, la navigation attend les données, et la page arrive complète.



La navigation attend le resolver ; la page est créée avec ses données.

2. La démonstration

Ouvrir une fiche avec l'onglet Réseau réglé sur « 3G lente » : la page apparaît vide, puis se remplit.

3. La correction

src/app/pages/product-detail/product-resolver.ts

import { inject } from '@angular/core';

```ts
import { inject } from '@angular/core';
import { RedirectCommand, ResolveFn, Router } from '@angular/router';
import { catchError, of } from 'rxjs';
import { Product } from '@shopflow/models';
import { ProductService } from '@shopflow/data-access';

/** Charge le produit avant d'afficher la fiche. */
export const productResolver: ResolveFn<Product> = (route) => {
  const router = inject(Router);
  const id = Number(route.paramMap.get('id'));
  return inject(ProductService).getById(id).pipe(
    catchError(() => of(new RedirectCommand(router.parseUrl('/catalogue')))),
  );
};
```

src/app/app.config.ts (extrait) (extrait)

```
provideRouter(routes, withComponentInputBinding()),
```

src/app/pages/product-detail/product-detail.ts

```
import { Component, inject, input } from '@angular/core';
import { CurrencyPipe, NgOptimizedImage } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Product } from '@shopflow/models';
import { CartService } from '@shopflow/data-access';
import { Note } from '@shopflow/ui';
import { ProductReviews } from './product-reviews';

@Component({
  selector: 'app-product-detail',
  imports: [CurrencyPipe, NgOptimizedImage, Note, ProductReviews, RouterLink],
  templateUrl: './product-detail.html',
})
export class ProductDetail {
  private readonly cart = inject(CartService);
  readonly produit = input.required<Product>();

  protected ajouter(p: Product): void {
    this.cart.add(p);
  }
}
```

4. Le résultat

La fiche n’apparaît qu’avec ses données ; le composant ne contient plus de requête, il reçoit le produit par une entrée.

14 h 25 – 15 h 00

Les signaux : un état partagé qui se met à jour seul

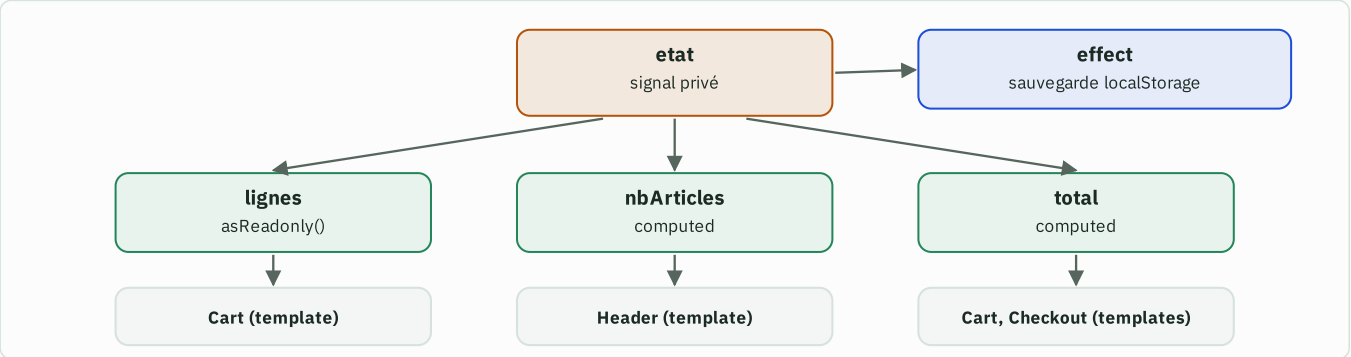
Module 3, « Signaux : définition, lecture seule, computed, effect ».

1. La notion

Objectif : gérer le panier avec des signaux : une valeur, des valeurs calculées, une sauvegarde automatique.

Pourquoi : le panier repose sur un `BehaviorSubject` : chaque lecture demande un abonnement, et le nombre d'articles et le total sont recalculés à la main.

Dans la réalité : l'en-tête, la page panier et la commande affichent le même panier. Avec un signal, chacun lit la valeur comme une variable, et l'affichage suit tout seul.



Le panier en signaux : un état privé, trois lectures, un effet. Chaque template se met à jour seul quand ce qu'il lit change.

2. La démonstration

libs/shared/data-access/src/lib/services/cart-service.ts (avant, extrait) (extrait)

```
@Service()
```

```
@Service()
export class CartService {
  private readonly lignes = new BehaviorSubject<CartLine[]>(this.lire());
  readonly lines$ = this.lignes.asObservable();
  readonly count$ = this.lines$.pipe(map((lignes) => lignes.reduce((n, l) => n + l.quantity, 0)));
  readonly total$ = this.lines$.pipe(map((lignes) => lignes.reduce((s, l) => s + l.price * l.quantity, 0)));

  add(product: Product): void {
    const lignes = this.lignes.value;
    // … calcul de « suivantes »
    this.publier(suivantes);
  }

  private publier(lignes: CartLine[]): void {
    this.lignes.next(lignes);
    localStorage.setItem(CLE_STOCKAGE, JSON.stringify(lignes));
  }
}
```

3. La correction

libs/shared/data-access/src/lib/services/cart-service.ts (après) `import { computed, effect, Service, signal`

```ts
import { computed, effect, Service, signal } from '@angular/core';
import { CartLine, Product, prixRemise } from '@shopflow/models';

/** Clé sous laquelle le panier est enregistré dans le localStorage du navigateur. */
const CLE_STOCKAGE = 'shopflow-panier';

function lire(): CartLine[] {
  const brut = localStorage.getItem(CLE_STOCKAGE);
  return brut ? JSON.parse(brut) : [];
}

/**
 * État du panier de l'utilisateur, en signaux.
 *
 * Les composants lisent `lignes()`, `nbArticles()` et `total()` ; ils ne modifient le panier
 * qu'au travers des méthodes de ce service.
 */
@Service()
export class CartService {
  private readonly etat = signal<CartLine[]>(lire());

  /** Lignes du panier (lecture seule). */
  readonly lignes = this.etat.asReadonly();

  /** Nombre total d'articles (somme des quantités). */
  readonly nbArticles = computed(() => this.etat().reduce((n, l) => n + l.quantity, 0));

  /** Montant total du panier, prix remisés compris. */
  readonly total = computed(() => this.etat().reduce((s, l) => s + l.price * l.quantity, 0));

  constructor() {
    effect(() => localStorage.setItem(CLE_STOCKAGE, JSON.stringify(this.etat())));
  }

  /**
   * Ajoute un exemplaire du produit ; si le produit est déjà présent, sa quantité augmente de 1.
   * @param product Produit à ajouter, tel que renvoyé par l'API
   */
  add(product: Product): void {
    this.etat.update((lignes) =>
      lignes.some((l) => l.productId === product.id)
        ? lignes.map((l) => (l.productId === product.id ? { ...l, quantity: l.quantity + 1 } : l))
        : [...lignes, { productId: product.id, title: product.title, price: prixRemise(product), quantity: 1,
thumbnail: product.thumbnail }],
    );
  }

  /**
   * Modifie la quantité d'une ligne. Une quantité nulle ou négative retire la ligne.
   * @param productId Identifiant du produit
```

```
   * @param quantity Nouvelle quantité
   */
  setQuantity(productId: number, quantity: number): void {
    this.etat.update((lignes) =>
      quantity <= 0 ? lignes.filter((l) => l.productId !== productId) : lignes.map((l) => (l.productId === productId ? { ...l, quantity } : l)),
    );
  }

  /**
   * Retire une ligne du panier.
   * @param productId Identifiant du produit à retirer
   */
  remove(productId: number): void {
    this.etat.update((lignes) => lignes.filter((l) => l.productId !== productId));
  }

  /** Vide le panier (après une commande). */
  clear(): void {
    this.etat.set([]);
  }
}
```

4. Le résultat

Vérification	Résultat
Trois ajouts depuis le catalogue	En-tête : « Panier (3) »
« + » dans le panier	« Panier (4) », total mis à jour
Rechargement de la page	« Panier (4) » : l'effet a sauvegardé le panier

15 h 00 – 15 h 35

Signaux et RxJS : passer de l’un à l’autre

Module 3, « Lien avec les observables : toSignal, toObservable ».

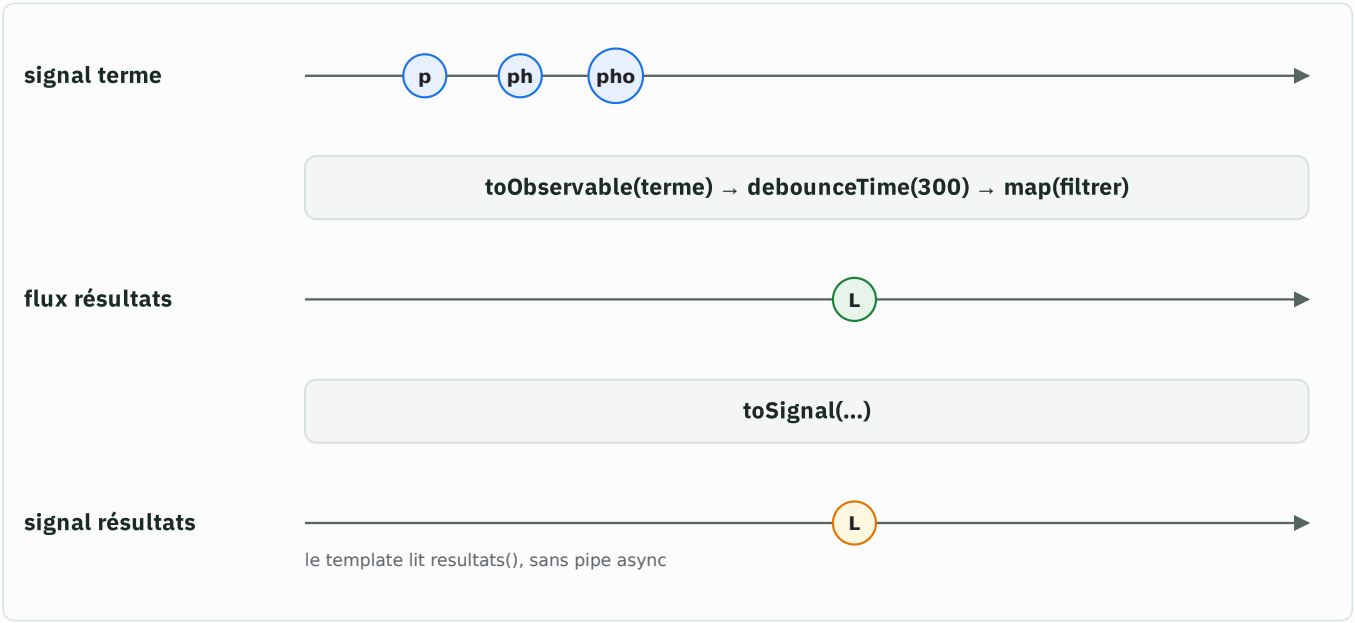
1. La notion

Objectif : utiliser des signaux dans le catalogue, en gardant RxJS là où il est utile (attendre, annuler).

Pourquoi : les signaux ne savent pas attendre la fin d’une frappe ni annuler une requête ; RxJS si. Il faut passer de l’un à l’autre.

Dans la réalité : dans une application réelle, l’état de l’écran est en signaux, et les requêtes au serveur restent des observables : `toSignal` et `toObservable` font le lien.

> **En deux mots**
>
> `toObservable` transforme une valeur en **flux** (pour pouvoir attendre et annuler) ; `toSignal` transforme le flux en **valeur** (pour l’afficher simplement).



toObservable transforme le signal en flux pour pouvoir attendre ; toSignal transforme le flux en signal pour l’afficher.

2. La démonstration

Le catalogue mélange `FormControl`, `combineLatest` et pipe `async` : trois façons de faire pour un seul écran.

3. La correction

src/app/pages/catalogue/catalogue.ts

```
import { Component, computed, inject, signal } from '@angular/core';
import { toObservable, toSignal } from '@angular/core/rxjs-interop';
```

```ts
import { debounceTime, distinctUntilChanged, map } from 'rxjs';
import { Product } from '@shopflow/models';
import { CartService, ProductService } from '@shopflow/data-access';
import { ProductCard } from '@shopflow/ui';

type Tri = 'nom' | 'prix-croissant' | 'prix-decroissant';

function filtrerEtTrier(produits: Product[], terme: string, categorie: string, tri: Tri): Product[] {
  const filtres = produits.filter((p) => p.title.toLowerCase().includes(terme) && (!categorie || p.category
=== categorie));
  return [...filtres].sort((a, b) =>
    tri === 'nom' ? a.title.localeCompare(b.title) : tri === 'prix-croissant' ? a.price - b.price : b.price -
a.price,
  );
}

@Component({
  selector: 'app-catalogue',
  imports: [ProductCard],
  templateUrl: './catalogue.html',
})
export class Catalogue {
  private readonly productService = inject(ProductService);
  protected readonly panier = inject(CartService);

  protected readonly page = toSignal(this.productService.getAll());
  protected readonly categories = toSignal(this.productService.getCategories(), { initialValue: [] });

  protected readonly saisie = signal('');
  protected readonly categorie = signal('');
  protected readonly tri = signal<Tri>('nom');

  private readonly terme = toSignal(
    toObservable(this.saisie).pipe(debounceTime(300), map((t) => t.trim().toLowerCase()),
distinctUntilChanged()),
    { initialValue: '' },
  );

  protected readonly produits = computed(() =>
    filtrerEtTrier(this.page()?.products ?? [], this.terme(), this.categorie(), this.tri()),
  );

  protected onRecherche(event: Event): void {
    this.saisie.set((event.target as HTMLInputElement).value);
  }

  protected onCategorie(event: Event): void {
    this.categorie.set((event.target as HTMLSelectElement).value);
  }

  protected onTri(event: Event): void {
    this.tri.set((event.target as HTMLSelectElement).value as Tri);
```

```
  }
}
```

src/app/pages/catalogue/catalogue.html

<h1>Catalogue</h1>

```
<h1>Catalogue</h1>
@if (page(); as page) {
  <p class="sous-titre">{{ page.total }} produits</p>
  <div class="barre-outils">
    <input class="champ recherche" type="search" placeholder="Rechercher un produit…"
(input)="onRecherche($event)" />
    <select class="champ" (change)="onCategorie($event)">
      <option value="">Toutes les catégories</option>
      @for (c of categories(); track c.slug) {
        <option [value]="c.slug">{{ c.name }}</option>
      }
    </select>
    <select class="champ" (change)="onTri($event)">
      <option value="nom">Trier par nom</option>
      <option value="prix-croissant">Prix croissant</option>
      <option value="prix-decroissant">Prix décroissant</option>
    </select>
  </div>
  <div class="grille">
    @for (p of produits(); track p.id) {
      <sf-product-card [product]="p" [prioritaire]="$index < 4" (ajout)="panier.add($event)" />
    } @empty {
      <p class="info">Aucun produit ne correspond à votre recherche.</p>
    }
  </div>
} @else {
  <p class="info">Chargement du catalogue…</p>
}
```

4. Le résultat

Même comportement : un seul calcul pendant la frappe, mais un template sans pipe `async`, qui lit des signaux.

15 h 35 – 15 h 50 : pause

15 h 50 – 17 h 00

TP Cuisto, jour 2

Objectif du TP

À la fin du TP : la recherche de Cuisto n'envoie plus une requête par lettre tapée, les pages du compte sont réservées aux personnes connectées, et le planning de la semaine est géré avec des signaux.

Pourquoi : chaque requête inutile coûte (serveur, données mobiles) et une réponse lente peut afficher d'anciens résultats ; une page privée ne doit pas s'ouvrir sans connexion ; les signaux mettent l'affichage à jour sans abonnement à gérer.

À rendre : le dépôt avec l'étiquette `jour-2` et la partie « Jour 2 » du rapport.

Copier les commandes depuis la version HTML de ce document (bouton **Copier**) ou depuis le fichier de commandes, jamais depuis le PDF. Sous Windows, utiliser **Git Bash**.

Point de départ : le Cuisto repris ce matin (`cuisto-debut-jour-2.zip`), avec Nx et les trois bibliothèques déjà en place. Il tourne sur le port 4201 (`npm start -- --port 4201`).

1. Une seule requête par recherche (25 min)

À obtenir : en tapant « pasta » rapidement, une seule requête part ; une réponse lente ne remplace plus une réponse plus récente.

Constat : F12, onglet **Réseau**, filtre `search` , taper « pasta » : une requête par lettre.

Fichiers : `src/app/pages/recipes/recipes.ts` et `recipes.html` .

À faire : au lieu de lancer une recherche à chaque frappe, envoyer chaque frappe dans un flux et transformer ce flux : attendre la fin de la frappe (`debounceTime(300)`), ignorer un terme identique au précédent (`distinctUntilChanged()`), lancer la recherche en abandonnant la précédente (`switchMap`). Le template affiche le résultat avec le pipe `async` .

Indice : les frappes peuvent être poussées dans un `Subject<string>` depuis la méthode `onRecherche` ; le flux des résultats part de ce `Subject` .

> **Vérifier** : une seule ligne `search` dans l'onglet Réseau pour « pasta ».

2. Protéger les pages du compte (20 min)

À obtenir : sans être connecté, `http://localhost:4201/compte/favoris` renvoie vers la page de connexion.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json)

```
npx nx g @schematics/angular:guard \
  --name=auth --project=cuisto --path=src/app/guards \
  --implements=CanActivateChild --skip-tests
```

À faire : dans `src/app/guards/auth-guard.ts`, renvoyer `true` si une session existe, sinon une redirection vers `/connexion` (`router.createUrlTree(['/connexion'])`). Puis, dans `src/app/app.routes.ts`, ajouter `canActivateChild: [authGuard]` à la route `compte`.

Indice : `AuthService` expose la session sous forme d'observable (`user$`) : un guard peut renvoyer un observable de `true` ou d'`UrlTree`, à condition de ne prendre que la première valeur.

Vérifier : déconnecté, « Mon compte » mène à la connexion ; connecté avec `emilys` / `emilyspass`, la page s'ouvre.

3. Le planning en signaux (20 min)

À obtenir : le planning est un signal, le nombre de repas une valeur calculée, et la page Planning se met à jour sans pipe `async`.

Fichiers : `libs/recettes/data-access/src/lib/planning-service.ts`, `src/app/account/week-planning.ts` et `.html`.

À faire : remplacer le `BehaviorSubject` par un `signal` exposé en lecture seule (`asReadonly()`) ; calculer le nombre de repas avec `computed()` ; dans `ajouter` et `retirer`, créer un **nouvel** objet avec `update()` au lieu de modifier l'ancien ; dans la page, lire `planning()` directement.

Indice : un `push` sur le tableau existant ne prévient pas le signal : sa valeur reste le même objet. Il faut lui donner une copie modifiée.

Vérifier : sur une fiche, « Ajouter au planning » ; dans la page Planning, la recette et le nombre de repas apparaissent aussitôt.

4. Rendre (5 min)

À obtenir : le dépôt étiqueté et le rapport complété.

Terminal — dans cuisto/ (racine du projet, là où se trouve package.json) git add -A

```
git add -A
git commit -m "Jour 2"
git tag jour-2
git push --follow-tags
```

Pour aller plus loin (facultatif)

À faire seulement si les étapes précédentes sont terminées. Chaque point donne des points bonus.

- Favoris et Mes recettes sans abonnements imbriqués (`forkJoin`).
- Après connexion, revenir à la page demandée ; réserver `/admin` aux administrateurs (guard par rôle).
- Charger la recette et son auteur avant d'afficher la fiche (resolver).

- Favoris en signaux, sauvegarde par `effect()` ; vérifier le graphe des signaux dans Angular DevTools.
- Regrouper la configuration dans une fonction `provideCuisto()` ; deux minuteurs indépendants sur la fiche.

Si le temps le permet

Pour aller plus loin

Les commandes du découpage en bibliothèques

2. Découper ShopFlow

Étape 1 : générer les trois bibliothèques.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx g @nx/angular:library

```
npx nx g @nx/angular:library --directory=libs/shared/models --name=models --importPath=@shopflow/models --
unitTestRunner=none --prefix=sf --skipModule
npx nx g @nx/angular:library --directory=libs/shared/data-access --name=data-access --
importPath=@shopflow/data-access --unitTestRunner=none --prefix=sf
npx nx g @nx/angular:library --directory=libs/shared/ui --name=ui --importPath=@shopflow/ui --
unitTestRunner=none --prefix=sf
```

Option	Rôle
`--directory`	Dossier de la bibliothèque
`--name`	Nom du projet dans Nx
`--importPath`	Alias utilisé dans les `import` : `@shopflow/ui` au lieu d'un chemin relatif
`--unitTestRunner=none`	Pas de configuration de test pour l'instant (module 7)
`--prefix=sf`	Sélecteurs en `sf-…`

Terminal : shopflow

```
$ npx nx g @nx/angular:library --directory=libs/shared/data-access --name=data-access --
importPath=@shopflow/data-access --unitTestRunner=none --prefix=sf
 NX  Generating @nx/angular:library
CREATE .prettierignore
UPDATE .vscode/extensions.json
UPDATE package.json
UPDATE .gitignore
CREATE libs/shared/data-access/project.json
CREATE libs/shared/data-access/README.md
CREATE libs/shared/data-access/tsconfig.json
CREATE libs/shared/data-access/tsconfig.lib.json
CREATE libs/shared/data-access/src/index.ts
CREATE tsconfig.base.json
UPDATE tsconfig.json
UPDATE nx.json
CREATE libs/shared/data-access/src/lib/data-access/data-access.css
CREATE libs/shared/data-access/src/lib/data-access/data-access.ts
CREATE libs/shared/data-access/src/lib/data-access/data-access.html
```

La génération de data-access : la première bibliothèque crée aussi tsconfig.base.json.

tsconfig.base.json (extrait) (extrait) "paths": {

```
"paths": {
  "@shopflow/data-access": ["./libs/shared/data-access/src/index.ts"],
  "@shopflow/ui": ["./libs/shared/ui/src/index.ts"],
  "@shopflow/models": ["./libs/shared/models/src/index.ts"]
}
```

Chaque bibliothèque contient un composant d'exemple : on le supprime.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) rm -rf libs/shared/models/src/lib/*

```
rm -rf libs/shared/models/src/lib/* libs/shared/data-access/src/lib/* libs/shared/ui/src/lib/*
```

Étape 2 : déplacer le code. `git mv` déplace en conservant l'historique des fichiers.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) mkdir -p

```
mkdir -p libs/shared/data-access/src/lib/services libs/shared/ui/src/lib/product-card
git mv src/app/models/* libs/shared/models/src/lib/
git mv src/app/services/* libs/shared/data-access/src/lib/services/
git mv src/app/pages/catalogue/product-card.html libs/shared/ui/src/lib/product-card/
git mv src/app/pages/catalogue/product-card.ts libs/shared/ui/src/lib/product-card/
```

Étape 3 : déclarer ce que chaque bibliothèque expose. Le fichier `index.ts` est l'API publique : ce qui n'y est pas exporté reste interne.

libs/shared/models/src/index.ts export * from './lib/product';

```
export * from './lib/product';
export * from './lib/cart';
export * from './lib/user';
export * from './lib/order';
```

libs/shared/data-access/src/index.ts export * from

```
export * from './lib/services/product-service';
export * from './lib/services/cart-service';
export * from './lib/services/auth-service';
export * from './lib/services/order-service';
export * from './lib/services/auth-interceptor';
```

libs/shared/ui/src/index.ts export * from

```
export * from './lib/product-card/product-card';
```

Étape 4 : remplacer les chemins relatifs par les alias, dans l'application et dans `data-access`. VS Code le propose quand on ouvre un fichier en erreur ; on peut aussi chercher `/models/` et `/services/` dans tout le projet.

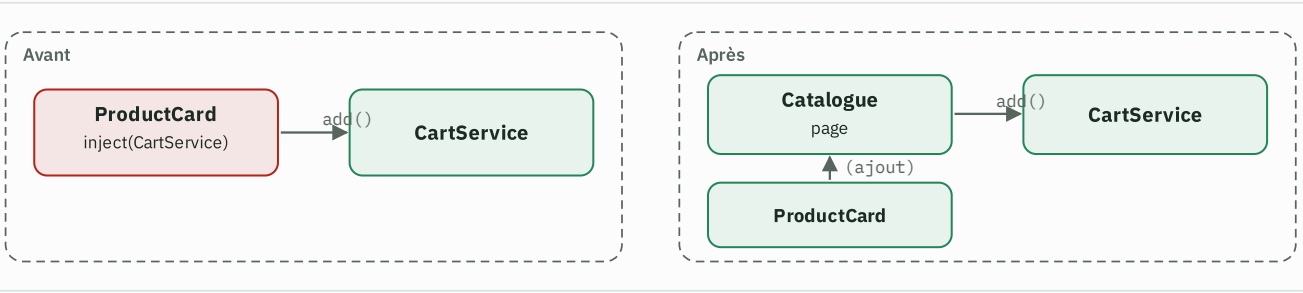
src/app/pages/orders/orders.ts (avant, extrait) (extrait) — import { Order } from

```ts
import { Order } from '../../models/order';
import { User } from '../../models/user';
import { AuthService } from '../../services/auth-service';
import { OrderService } from '../../services/order-service';
```

src/app/pages/orders/orders.ts (après, extrait) (extrait) — import { Order, User } from

```ts
import { Order, User } from '@shopflow/models';
import { AuthService, OrderService } from '@shopflow/data-access';
```

Étape 5 : rendre la carte indépendante du panier. Elle reçoit un produit et signale le clic ; c'est la page qui décide quoi en faire.



La carte émet un événement `ajout` ; la page Catalogue le reçoit et appelle le panier.

libs/shared/ui/src/lib/product-card/product-card.ts — import { Component, input, output } from

```ts
import { Component, input, output } from '@angular/core';
import { CurrencyPipe, DecimalPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Product, prixRemise } from '@shopflow/models';

@Component({
  selector: 'sf-product-card',
  imports: [CurrencyPipe, DecimalPipe, RouterLink],
  templateUrl: './product-card.html',
})
export class ProductCard {
  readonly product = input.required<Product>();
  readonly ajout = output<Product>();

  protected prixFinal(): number {
    return prixRemise(this.product());
  }

  protected ajouter(): void {
    this.ajout.emit(this.product());
  }
}
```

src/app/pages/catalogue/catalogue.ts — import { Component, inject } from

```ts
import { Component, inject } from '@angular/core';
import { AsyncPipe } from '@angular/common';
```

```ts
import { Product } from '@shopflow/models';
import { CartService, ProductService } from '@shopflow/data-access';
import { ProductCard } from '@shopflow/ui';

type Tri = 'nom' | 'prix-croissant' | 'prix-decroissant';

@Component({
  selector: 'app-catalogue',
  imports: [AsyncPipe, ProductCard],
  templateUrl: './catalogue.html',
})
export class Catalogue {
  private readonly productService = inject(ProductService);
  protected readonly panier = inject(CartService);

  protected readonly page$ = this.productService.getAll();
  protected readonly categories$ = this.productService.getCategories();
  protected recherche = '';
  protected categorie = '';
  protected tri: Tri = 'nom';

  protected produitsAffiches(produits: Product[]): Product[] {
    const terme = this.recherche.trim().toLowerCase();
    const filtres = produits.filter(
      (p) => p.title.toLowerCase().includes(terme) && (!this.categorie || p.category === this.categorie),
    );
    return [...filtres].sort((a, b) =>
      this.tri === 'nom' ? a.title.localeCompare(b.title) : this.tri === 'prix-croissant' ? a.price - b.price
: b.price - a.price,
    );
  }

  protected onRecherche(event: Event): void {
    this.recherche = (event.target as HTMLInputElement).value;
  }

  protected onCategorie(event: Event): void {
    this.categorie = (event.target as HTMLSelectElement).value;
  }

  protected onTri(event: Event): void {
    this.tri = (event.target as HTMLSelectElement).value as Tri;
  }
}
```

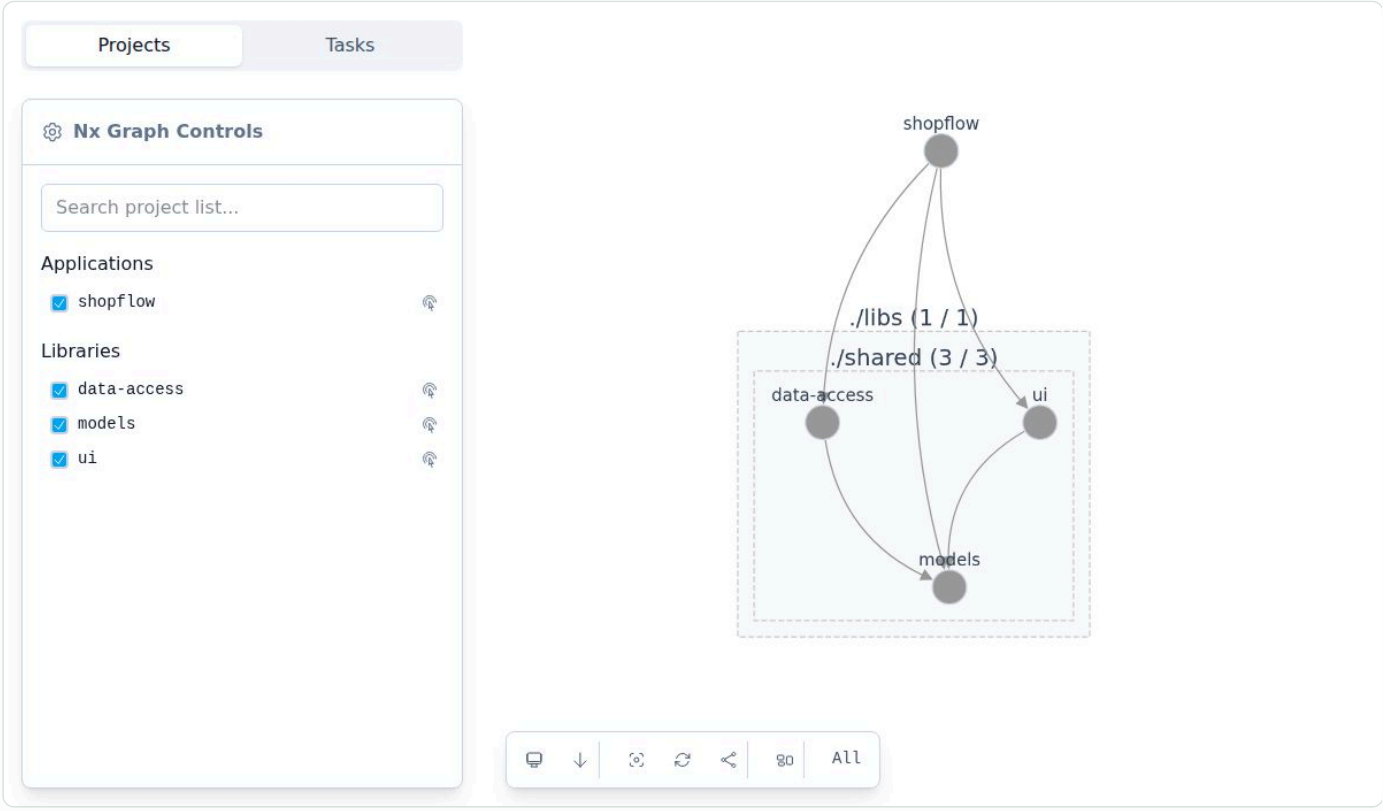
src/app/pages/catalogue/catalogue.html (extrait) (extrait) @for (p of

```
@for (p of produitsAffiches(page.products); track $index) {
  <sf-product-card [product]="p" (ajout)="panier.add($event)" />
}
```

Étape 6 : vérifier. `npx nx build` doit réussir, et le bouton « Ajouter au panier » fonctionner comme avant. Puis on affiche le graphe :

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx graph

```
npx nx graph
```



Le graphe : l'application dépend des trois bibliothèques ; ui et data-access dépendent de models ; ui ne dépend pas de data-access.

Terminal : shopflow

```
$ npx nx show projects
data-access
models
ui
shopflow
$ # après une modification dans libs/shared/ui
npx nx show projects --affected
ui
shopflow
$ # après une modification dans libs/shared/models
npx nx show projects --affected
models
data-access
shopflow
ui
```

Les projets touchés : une modification dans ui ne concerne que ui et l'application ; une modification dans models concerne tout le monde.

Pour finir, on donne une étiquette (`tags`) à chaque projet, dans son `project.json` : `type:app` , `type:data-access` , `type:ui` , `type:models` . Ces étiquettes servent à la règle ESLint `@nx/enforce-module-boundaries` , qui refuse par exemple qu'une bibliothèque `ui` importe `data-access` .
ShopFlow n'a pas d'ESLint : on se contente des étiquettes, la règle est un prolongement possible.

libs/shared/ui/project.json (extrait) (extrait) "tags": ["type:ui"],

```
"tags": ["type:ui"],
```

3. Gérer les bibliothèques : ajouter un composant, un service, une bibliothèque (8 min)

Une fois le dépôt découpé, on ne crée plus les fichiers à la main : on demande à un **générateur**. Il place le fichier au bon endroit, avec le bon préfixe (`sf-`), et pour un composant il l'ajoute aux exports de la bibliothèque. Chaque action se fait en ligne de commande **ou** dans Nx Console ; les commandes se lancent dans `shopflow/` .

Ajouter un composant dans la bibliothèque `ui` (exemple : des étoiles de notation)

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx g @nx/angular:component

```
npx nx g @nx/angular:component libs/shared/ui/src/lib/etoiles/etoiles --export --skipTests --style=none
```

Dans Nx Console

1. Icône **Nx** > **Common Nx Commands** > **Generate (UI)**.
2. Taper `component` , choisir **@nx/angular - component**.
3. Champ **path** : `libs/shared/ui/src/lib/etoiles/etoiles` ; cocher **export** ; dans **Show all options**, cocher **skipTests** et choisir **style** : `none` .
4. Bouton **Generate**, en haut à droite.

Résultat obtenu CREATE

```
CREATE libs/shared/ui/src/lib/etoiles/etoiles.ts
CREATE libs/shared/ui/src/lib/etoiles/etoiles.html
UPDATE libs/shared/ui/src/index.ts
```

Pour l'utiliser dans l'application : `import { Etoiles } from '@shopflow/ui';` , l'ajouter au tableau `imports` du composant, puis écrire `<sf-etoiles />` dans le template.

Ajouter un service dans la bibliothèque `data-access` (exemple : des favoris). Nx n'a pas de générateur de service pour Angular : on utilise celui d'Angular, appelé par Nx.

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx g @schematics/angular:service

```
npx nx g @schematics/angular:service --name=favoris-service --project=data-access --path=libs/shared/data-access/src/lib/services --skip-tests
```

Dans Nx Console

1. **Generate (UI)**, taper `service` , choisir **@schematics/angular - service**.
2. Champs **name** : `favoris-service` ; **project** : `data-access` ; **path** : `libs/shared/data-access/src/lib/services` ; cocher **skipTests**.
3. Bouton **Generate**.

Résultat obtenu CREATE

```
CREATE libs/shared/data-access/src/lib/services/favoris-service.ts
```

À ne pas oublier

Le générateur de service **n'ajoute pas l'export** : ajouter à la main, à la fin de `libs/shared/data-access/src/index.ts`, la ligne `export * from './lib/services/favoris-service';`. Sans elle, l'application ne peut pas importer `FavorisService` depuis `@shopflow/data-access`.

Créer, déplacer, supprimer une bibliothèque

Je veux…	Commande (dans shopflow/)	Dans Nx Console : Generate (UI), puis…
Créer une bibliothèque	`npx nx g @nx/angular:library --directory=libs/shared/utils --name=utils --importPath=@shopflow/utils --unitTestRunner=none --prefix=sf`	**@nx/angular - library**
Voir ce qui serait créé, sans rien écrire	Ajouter `--dry-run` à la commande	—
Renommer ou déplacer une bibliothèque	`npx nx g @nx/workspace:move --projectName=utils --destination=libs/shared/outils --newProjectName=outils --importPath=@shopflow/outils`	**@nx/workspace - move**
Supprimer une bibliothèque	`npx nx g @nx/workspace:remove --projectName=outils`	**@nx/workspace - remove**

La création d'une bibliothèque ajoute aussi son alias dans `tsconfig.base.json` (`"@shopflow/utils": ["./libs/shared/utils/src/index.ts"]`) ; le déplacement et la suppression mettent cet alias à jour. Lors de notre essai, `npx nx show projects` listait `utils`, puis `outils`, puis plus rien.

Ce que touche une modification : après l'ajout du composant `etoiles`, sans rien enregistrer dans Git :

Terminal — dans shopflow/ (racine du projet, là où se trouve package.json) npx nx show projects --affected

```
npx nx show projects --affected
```

Résultat obtenu : `ui shopflow`. La bibliothèque modifiée et l'application qui l'utilise ; ni `models` ni `data-access`, qui ne dépendent pas de `ui`.

La règle à retenir

- On importe toujours par l'alias (`@shopflow/ui`), jamais par un chemin relatif vers `libs/…`.
- Un composant ou un service partagé va dans une bibliothèque ; ce qui n'appartient qu'à une page reste dans l'application.
- Avant de lancer un générateur inconnu, `--dry-run` montre ce qu'il va créer.

Voir les signaux dans Angular DevTools

6. Voir les signaux dans Angular DevTools (10 min)

Le besoin : quand une valeur ne se met pas à jour, il faut savoir de quoi elle dépend. Angular DevTools affiche le **graphe des signaux** d'un composant : chaque signal, chaque `computed`, chaque effet, et les liens entre eux, mis à jour en direct.

On commence par donner un nom à chaque signal du panier : sans nom, le graphe affiche des nœuds anonymes.

libs/shared/data-access/src/lib/services/cart-service.ts (extrait) (extrait) — private readonly etat =

```
private readonly etat = signal<CartLine[]>(lire(), { debugName: 'panier' });
readonly nbArticles = computed(() => this.etat().reduce((n, l) => n + l.quantity, 0), { debugName:
'nbArticles' });
readonly total = computed(() => this.etat().reduce((s, l) => s + l.price * l.quantity, 0), { debugName:
'total' });

constructor() {
  effect(() => localStorage.setItem(CLE_STOCKAGE, JSON.stringify(this.etat())), { debugName: 'sauvegarde' });
}
```

Terminal : shopflow — ouvrir http://localhost:4200, puis F12, onglet « Angular » — npm start

```
npm start
```

Les manipulations :

1. Dans l'onglet **Angular**, sous-onglet **Components**, sélectionner `app-header` dans l'arbre (ou cliquer sur l'en-tête avec l'outil de sélection).
2. Dans le panneau des propriétés, cliquer sur **Show Signal Graph**.
3. Lire le graphe : le signal `panier`, les deux `computed` qui en dépendent (`nbArticles`, `total`), et l'effet `sauvegarde`.
4. Ajouter un produit au panier : les valeurs du graphe changent en direct.
5. Sur le nœud `panier`, choisir **Watch signal** : chaque mise à jour est écrite dans la console.

> **Bon à savoir**
>
> Le graphe ne fonctionne qu'en mode développement (`npm start`), avec Angular 19 ou plus récent. Dans les premières versions de l'extension, il fallait l'activer dans les réglages (icône d'engrenage, option « Signal Graph ») ; le libellé des boutons peut varier selon la version installée. Penser à capturer l'écran pendant la formation pour le support.

> **En deux mots**
>
> Le graphe des signaux, c'est le **plan de la tuyauterie** : on voit d'où vient chaque valeur et ce qui coule vers où.
>
> 1. `panier` est le réservoir : c'est le seul signal qu'on modifie.
> 2. `nbArticles` et `total` sont des robinets branchés dessus : ils se recalculent quand `panier` change.
> 3. `sauvegarde` est un effet branché sur `panier` : il recopie le panier dans le localStorage.

Le piège : un second panier

7. Le piège : un second panier (5 min)

Démonstration à faire après le TP, puis à défaire. On ajoute une seule ligne dans la page panier :

src/app/pages/cart/cart.ts (extrait, à retirer ensuite) (extrait) `@Component({`

```
@Component({
  selector: 'app-cart',
  imports: [CurrencyPipe, RouterLink],
  providers: [CartService],
  templateUrl: './cart.html',
})
```

Terminal : shopflow — ajouter un produit depuis le catalogue, ouvrir /panier, cliquer sur « + » `npm start`

```
npm start
```

Vérification	Résultat obtenu
En-tête après l'ajout depuis le catalogue	« Panier (1) »
Clic sur « + » dans la page panier	Le total de la page change
En-tête après ce clic	Toujours « Panier (1) » : l'en-tête lit l'autre panier

En deux mots

Deux instances, c'est **deux caisses enregistreuses** dans le même magasin : on encaisse sur l'une, l'écran de l'autre ne bouge pas.

1. Sans `providers`, la page panier et l'en-tête reçoivent l'instance de la racine : un seul panier.
2. Avec `providers: [CartService]`, la page panier crée la sienne en arrivant (elle relit le localStorage, d'où le même contenu au départ).
3. Ensuite, chaque panier vit sa vie : l'en-tête ne voit pas les changements faits dans la page.

À retenir

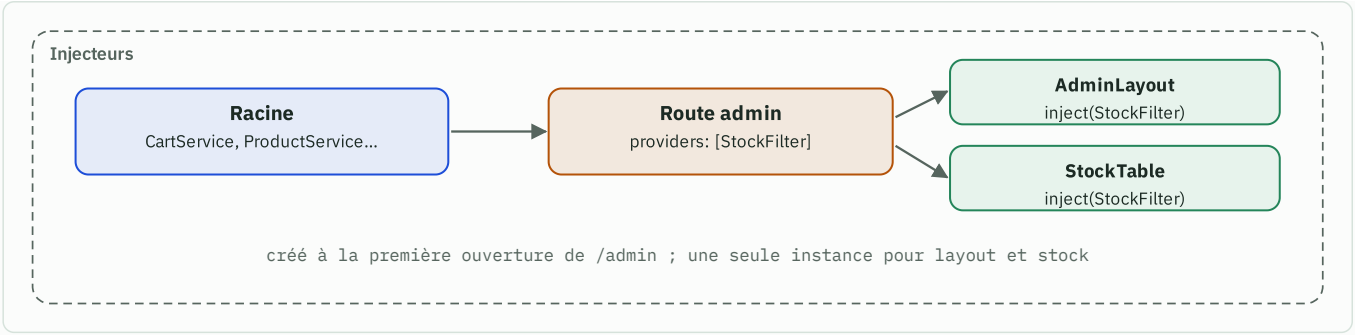
Un service à état partagé (panier, utilisateur connecté) est fourni **une seule fois**, à la racine. On le met dans `providers` d'un composant ou d'une route seulement quand on veut **une instance à part** : par exemple un filtre propre à l'administration (séquence 12) ou un minuteur par composant (TP Cuisto). Retirer la ligne `providers` après la démonstration.

Un service fourni par une route

3. Un service fourni par une route (10 min)

Le besoin : l'administration veut filtrer le tableau du stock, et garder ce filtre quand on passe du tableau de bord au stock. Ce filtre ne concerne que l'administration : inutile de le créer, ni même de le télécharger, pour un visiteur du catalogue.

Avec les NgModules, on aurait mis ce service dans le tableau `providers` du module de l'administration. Sans module, c'est la **route** qui le fournit : toutes les pages enfants de `admin` reçoivent la même instance, distincte du reste de l'application.



La route admin a son propre injecteur : StockFilter n'existe que là.

src/app/admin/stock-filter.ts `import { computed, Injectable, signal }`

```ts
import { computed, Injectable, signal } from '@angular/core';

/** Filtre du tableau du stock, propre à l'espace d'administration. */
@Injectable()
export class StockFilter {
  readonly terme = signal('');
  readonly seulementFaible = signal(false);
  readonly actif = computed(() => this.terme() !== '' || this.seulementFaible());

  constructor() {
    console.log('StockFilter : instance créée');
  }
}
```

src/app/admin/admin.routes.ts `import { Routes } from '@angular/router';`

```ts
import { Routes } from '@angular/router';
import { adminGuard } from '../guards/admin-guard';
import { AdminLayout } from './admin-layout';
import { AdminDashboard } from './admin-dashboard';
import { StockTable } from './stock-table';
import { StockFilter } from './stock-filter';

export const adminRoutes: Routes = [
  {
    path: '',
    component: AdminLayout,
    canActivateChild: [adminGuard],
    providers: [StockFilter],
    children: [
      { path: '', component: AdminDashboard },
      { path: 'stock', component: StockTable },
    ],
  },
];
```

src/app/admin/stock-table.ts `import { Component, computed, inject }`

```ts
import { Component, computed, inject } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { toSignal } from '@angular/core/rxjs-interop';
```

```ts
import { ProductService } from '@shopflow/data-access';
import { BadgeStock } from '@shopflow/ui';
import { StockFilter } from './stock-filter';

@Component({
  selector: 'app-stock-table',
  imports: [BadgeStock, CurrencyPipe],
  templateUrl: './stock-table.html',
})
export class StockTable {
  protected readonly filtre = inject(StockFilter);
  private readonly page = toSignal(inject(ProductService).getAll());
  protected readonly lignes = computed(() => {
    const terme = this.filtre.terme().toLowerCase();
    return (this.page()?.products ?? []).filter(
      (p) => p.title.toLowerCase().includes(terme) && (!this.filtre.seulementFaible() || p.stock < 10),
    );
  });
}
```

src/app/admin/stock-table.html

```html
<p class="barre-outils">
  <input class="champ" type="search" placeholder="Filtrer les produits…" [value]="filtre.terme()"
(input)="filtre.terme.set($any($event.target).value)" />
  <label><input type="checkbox" [checked]="filtre.seulementFaible()"
(change)="filtre.seulementFaible.set($any($event.target).checked)" /> Stock faible seulement</label>
</p>
@if (lignes().length > 0) {
  <div class="table">
    <table class="tableau">
      <thead>
        <tr><th>Produit</th><th>Catégorie</th><th>Prix</th><th>Stock</th><th>Valeur</th><th>État</th></tr>
      </thead>
      <tbody>
        @for (p of lignes(); track p.id) {
          <tr>
            <td>{{ p.title }}</td>
            <td>{{ p.category }}</td>
            <td>{{ p.price | currency: 'EUR' }}</td>
            <td>{{ p.stock }}</td>
            <td>{{ p.price * p.stock | currency: 'EUR' }}</td>
            <td><sf-badge-stock [stock]="p.stock" /></td>
          </tr>
        }
      </tbody>
    </table>
  </div>
} @else {
  <p class="info">Aucun produit.</p>
}
```

src/app/admin/admin-layout.ts

```
import { Component, inject } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';
import { StockFilter } from './stock-filter';

@Component({
  selector: 'app-admin-layout',
  imports: [RouterLink, RouterLinkActive, RouterOutlet],
  templateUrl: './admin-layout.html',
})
export class AdminLayout {
  protected readonly filtre = inject(StockFilter);
}
```

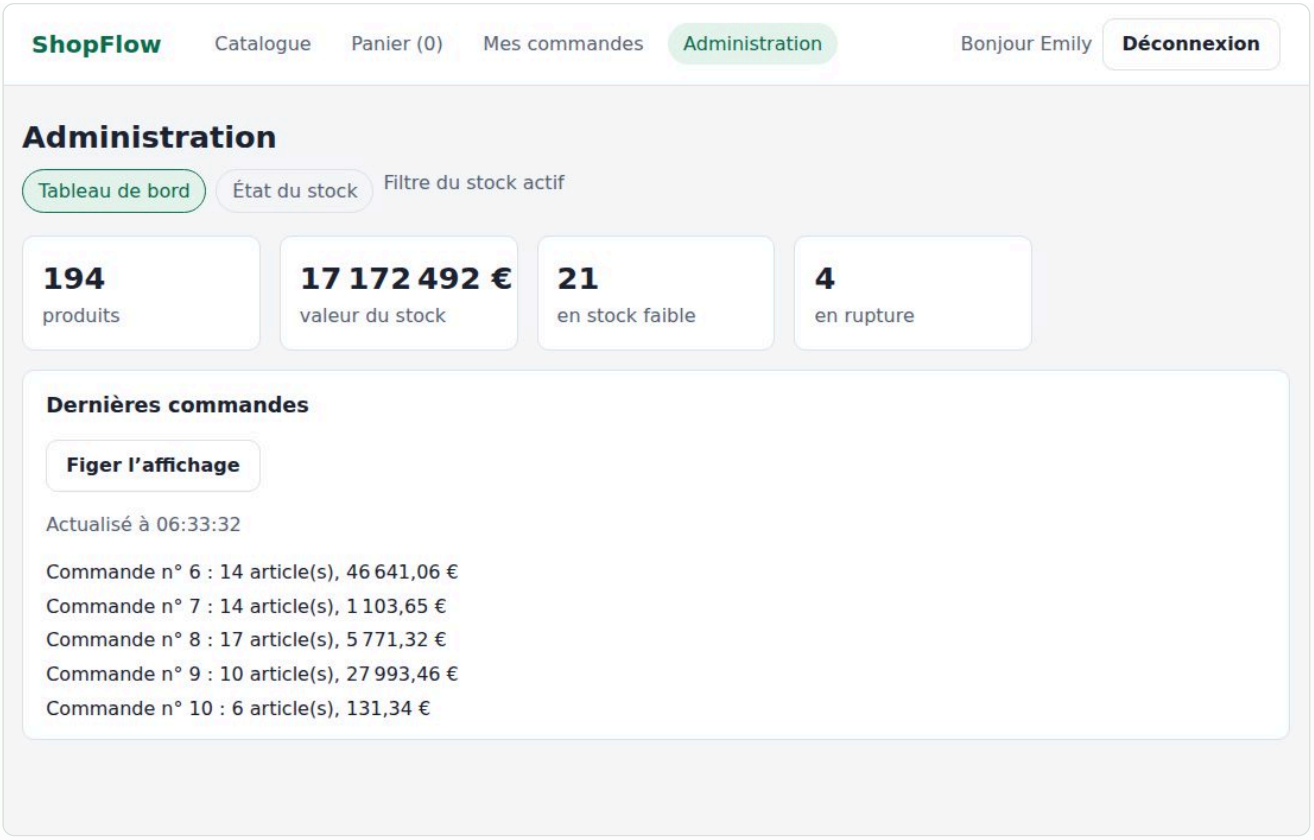
src/app/admin/admin-layout.html

```
<h1>Administration</h1>
<nav class="sous-nav">
  <a routerLink="/admin" routerLinkActive="actif" [routerLinkActiveOptions]="{ exact: true }">Tableau de bord</a>
  <a routerLink="/admin/stock" routerLinkActive="actif">État du stock</a>
  @if (filtre.actif()) {
    <span class="info">Filtre du stock actif</span>
  }
</nav>
<router-outlet />
```

Terminal : shopflow — se connecter (emilys / emilyspass), ouvrir http://localhost:4200/admin/stock, console ouverte (F12)

```
npm start
```

Vérification	Résultat obtenu
Message « StockFilter : instance créée » avant d'ouvrir l'administration	Aucun
À l'ouverture de /admin	Un message : l'instance est créée à ce moment-là
Filtre « table » dans le stock	2 lignes sur 194
Retour au tableau de bord	« Filtre du stock actif » dans le sous-menu : même instance
Catalogue, puis retour au stock	Le champ contient toujours « table » ; toujours une seule instance



Le tableau de bord indique que le filtre du stock est actif : layout et tableau partagent la même instance.

À savoir

- Une instance fournie par une route vit tant que l'application tourne : quitter `/admin` ne la détruit pas (le filtre est retrouvé au retour).
- Injecter `StockFilter` hors de l'administration (dans le catalogue, par exemple) provoque une erreur `NullInjectorError` : aucun injecteur ne le fournit.
- Le code du service est dans le fichier de l'administration, chargé à la demande : il n'alourdit pas le démarrage.

Exercice : les entrées de la carte

4. Exercice (10 min)

Énoncé

La note d'un produit (« ★ 4,5 ») est écrite deux fois : dans la carte et dans la fiche produit. Créer dans la bibliothèque `ui`, en ligne de commande ou avec Nx Console (Generate (UI) > @nx/angular - component), un composant `sf-note` qui reçoit la note et l'affiche, puis l'utiliser aux deux endroits. Vérifier avec `npx nx show projects --affected` quels projets sont touchés.

▶ **Afficher le corrigé**

Exercice : optimiser le tableau du stock

4. Fin du TP : optimiser le tableau du stock (10 min)

Énoncé

Le tableau de `/admin/stock` appelle deux méthodes pour chacune de ses 194 lignes et utilise `track $index`. Supprimer toutes les méthodes appelées par son template : un pipe pur `etatStock` pour l'état, un calcul direct pour la valeur, et `track` par identité.

▶ **Afficher le corrigé**